This article has been accepted for publication in IEEE Transactions on Mobile Computing. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TMC.2026.3745584

LLM Agent-Driven UAV Swarm for PoI Continual Monitoring in Hydraulic Metaverses

Yu Qiu, Min Chen, *Fellow, IEEE*, Weifa Liang, *Fellow, IEEE*,
Xing Xi, Haihan Zhang, Giancarlo Fortino, *Fellow, IEEE*, Fuxin Zhang, Hongwu Tang

***Abstract*—Monitoring points of interest (PoIs) representing water bodies and related infrastructures in hydraulic metaverses, with the help of unmanned aerial vehicles (UAVs), is central to protecting aquatic ecosystems and ensuring public safety. However, compared to traditional snapshot inspections, continual monitoring relies on UAV swarms to collect data from PoIs in a persistent manner, which must navigate complex operational constraints including limited battery capacity, strict user delay requirements, and dynamic weather conditions, while ensuring that learning-based detection models remain robust over time without suffering from catastrophic forgetting. To address this challenge, we propose an automatic hydraulic monitoring system across multiple task scenarios, utilizing a UAV swarm orchestrated by large language model (LLM) agents equipped with a suite of specialized tools. First, an environment perception tool is designed for the agent to observe dynamic global information, including regional weather and metaverse network topology. Building on this, a self-evolving swarm scheduling tool is developed to collect data from PoIs, where a feedback-based refinement module initially predicts the required number of UAVs and iteratively improves its decisions by leveraging verified feedback from a trajectory planning module operating in environments. To accommodate different learning paradigms across application scenarios, a dual-mode model update tool is further proposed to support both snapshot training with one-time full-data collection and continual learning through progressive data stream acquisition. Finally, experimental results demonstrate that our system outperforms baseline methods across multiple performance metrics, offering a continual monitoring system for the hydraulic metaverse.**

***Index Terms*—Continual learning, Large language model, Agent, Trajectory planning, Hydraulic engineering, Metaverse**

I. Introduction

COMPREHENSIVE monitoring of critical points of interest (PoIs) in hydraulic engineering is essential, since structural degradation may cause sudden failures and declining water quality threatens ecosystems. These PoIs cover both infrastructures such as bridges and pipelines and environmental factors like water quality in a city as shown in Fig. 1(a). With the rapid advancement of digital twins and metaverse technologies, comprehensive PoI monitoring needs continuous synchronization between physical objects and their virtual representations [19], [27]. Meanwhile, users in hydraulic metaverses expect to access the latest information about structural health and water quality through mobile devices, necessitating adaptive monitoring that evolves with environmental conditions. To support high-fidelity continual monitoring services, unmanned aerial vehicles (UAVs) deployed as swarms have emerged as a transformative solution, due to their mobility, scalability, and ability to continually capture multimodal data even in dangerous or inaccessible locations [12], [33].

However, most current UAV-assisted monitoring schemes rely on static data collection and offline-trained models [8], [9], [44], [43], [32], [5], which cannot provide high-fidelity continual monitoring services [1]. Although continual monitoring through the periodic collection of data from PoIs and ongoing model training is designed to fulfill these requirements, its implementation via UAV swarms in hydraulic metaverse networks presents several key challenges, particularly under dynamic weather conditions. (1) Weather conditions impact the number of UAVs needed to monitor the PoIs; (2) adverse weather conditions, such as strong winds [35], [34] and high air density, restrict the flight trajectory of each UAV in the swarm; (3) the strict monitoring delay requirements of users constrain the degree of parallel trajectories of the swarm; and (4) fine-tuning a pre-trained detection model with newly collected data may lead to forgetting previously learned information, resulting in degraded service performance.

To illustrate weather-related research motivations, consider a scenario where a UAV with 4,500 J energy and 20 m/s speed monitors four 200 m grid cells under given weather conditions, with wind direction, wind speed, and air density marked at the corners of each cell. Using the UAV parameters from [45], together with Eq. (1) and Property 1 in Section III, we calculate the UAV energy consumption. As shown in Fig. 1(b)(2), flying 600 m from A to B to C to D takes 30 s and consumes 4044.04 J, which is within the UAV's energy budget and delay limit, allowing it to complete all PoI data collection. However, variations in air density or wind direction

Corresponding author: Min Chen (email: minchen@ieee.org).

Yu Qiu, Min Chen, and Xing Xi are with the School of Computer Science and Engineering, South China University of Technology, and with Pa zhou Laboratory, Guangzhou, P. R. China (email: csqiuyu@mail.scut.edu.cn).

Weifa Liang is with the Department of Computer Science, City University of Hong Kong, Hong Kong, P. R. China (email: weifa.liang@cityu.edu.hk).

Haihan Zhang is with the School of Computer Science and Engineering, Nanjing University, Nanjing, P. R. China.

Giancarlo Fortino is with the Department of Informatics, Modeling, Electronics, and Systems, University of Calabria, Rende (CS) 87036, Italy (email: g.fortino@unical.it).

Fuxin Zhang and Hongwu Tang are with the School of Marine Science and Engineering, South China University of Technology, Guangzhou, P. R. China.

[1]Conventional UAV monitoring is typically designed for one-time target detection from UAV-captured images. In contrast, metaverse monitoring requires the entire monitoring pipeline, including target detection, data collection, and continual model learning, which is required to be executed periodically so that the digital twin can synchronize with the physical system. Since the real-world environment may change between successive monitoring cycles (e.g., different weather conditions), the system operates dynamically, requiring adaptive UAV scheduling and trajectory planning.

© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Mobile Computing. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TMC.2026.3745584

may exceed its energy budget in Fig. 1(b)(3) and (4), requiring additional UAVs (Challenge 1). Inefficient UAV trajectories lead to higher energy consumption (Challenge 2), as shown in Fig. 1(b)(5), while stringent delay requirements (e.g., 20 s) demand greater swarm parallelism to meet delay constraints (Challenge 3), as illustrated in Fig. 1(b)(6).

The novelty of this paper lies in the exploration of continual PoI monitoring by a swarm of UAVs in a hydraulic metaverse, where dynamic weather conditions influence both the number and trajectories of UAVs, and new data from PoIs are collected at each time slot to support continual learning in learning-based detection models over a given time horizon. In addition, for continual monitoring requests from users, an LLM agent-driven monitoring system is proposed to autonomously perceive dynamic environments and plan the deployment of the UAV swarm for data collection [2], considering weather conditions and user delay requirements, while enabling the service model to improve service accuracy in both the snapshot mode (all data are collected in a single slot) and the continual mode (data are collected over multiple slots). The overall process described above is illustrated in Fig. 1(e).

The main contributions of this paper are as follows.

- We are the first to study a continual monitoring problem that integrates real-time data collection and model training under strict UAV battery and user delay constraints, while accounting for dynamic weather and catastrophic forgetting, and prove that the problem is NP-hard.
- To empower users with an automated solution to the problem, we design an LLM agent that autonomously collects data by employing a UAV swarm and continually trains the detection models using data collected over time, based on natural language-described and task-specific prompts.
- To enable the agent to perform tasks efficiently, we develop an environment perception tool for real-time weather sensing, a self-evolving swarm tool that refines UAV deployment decisions through corresponding trajectory feedback, and a dual-mode model update tool that supports both snapshot-based and continual learning.
- Experiments show that our system can reduce UAV count by up to 1.2×, lower energy consumption by 29.6%, reduce delay by 18.4%, and improve model accuracy by up to 5.89 percentage points in snapshot mode and 3.64 percentage points in continual mode compared to state-of-the-art baselines.

The rest of this article is organized as follows. Section II reviews related works. Section III introduces network, weather, energy, and delay models. Section IV proposes the LLM agent-driven monitoring system for continual monitoring and model learning. Section V evaluates the performance of the proposed system. Section VI concludes the paper.

[2]In addition, the proposed system achieves PoI coverage using the best feasible UAV-count solution found within the refinement budget via self-evolution-based iterative refinement while offering operational simplicity through natural language interaction, overcoming the limitations of both learning-based and traditional optimization methods.

II. Related Work

Unmanned aerial vehicle: Guo *et al.* [14] considered both flying and monitoring delays, and proposed approximation algorithms to find collaborative flight tours for K (≥ 1) UAVs in a disaster area, with the aim to minimize the time spent for PoI monitoring. Deng *et al.* [9] devised approximation algorithms to minimize the longest length of K tours by utilizing neighboring nodes. Xu *et al.* [44] studied the problem of scheduling K heterogeneous UAVs to monitor PoIs in a disaster zone so that the sum of monitoring rewards received from all UAVs is maximized, subject to energy capacity on each UAV. They devised a constant approximation algorithm for the problem. Jia *et al.* [17] developed an ant colony optimization algorithm to jointly optimize UAV trajectories, hovering positions, and flight speeds, with the aim to minimize both the maximum task completion time and the total energy consumption of all UAVs. Shan *et al.* [32] modeled the swerving and switching energy consumption of a flying UAV, and proposed an algorithm by reducing the problem to a traveling salesman problem. Li *et al.* [22] considered full and partial data collection from each PoI in a monitoring area, using an energy-constrained UAV, for which they developed approximate and heuristic solutions to the problems. Chang *et al.* [5] focused on minimizing the number of UAVs needed for IoT data collection, and proposed a 3-approximation algorithm that includes randomized rounding and dynamic programming techniques with a given delay budget.

Unlike the aforementioned approximation algorithms for UAV trajectory problems by exploring the combinatorial properties of the problems, recent studies adopt reinforcement learning (RL) based approaches to optimize UAV trajectories under dynamic environments. Gong *et al.* [13] employed multi-agent deep RL to optimize UAV trajectories without knowing ground users' traffic demands and spatial distribution, and used Bayesian optimization to improve learning efficiency. Wang *et al.* [38] proposed a deep RL based algorithm to jointly optimize discrete computation modes, continuous 3D trajectories, and resource assignment for task processing delay minimization. Oubbat *et al.* [26] combined deep RL for UAV flight paths and recharging schedules with a genetic algorithm for UGV patrol routes and differential evolution for RIS configuration. Alotaibi *et al.* [2] integrated multi-agent RL with game theory to optimize UAV energy management, RIS control, and task offloading decisions, featuring a RIS ON/OFF mechanism for energy conservation. Ameur *et al.* [3] applied multi-agent deep RL to optimize UAV and UGV movements for adaptive trajectory adjustment, and employed a softmax-based probabilistic caching strategy to mitigate redundant storage. Recently, LLMs have been increasingly adopted for network and UAV decision-making. For instance, Wu *et al.* [40] proposed NetLLM, a general framework that adapts LLMs to a wide range of networking tasks through multi-modal processing and task-specific reasoning, providing a foundation for SLA-aware intelligent network management. Building upon their work, Wu *et al.* [42] leveraged in-context learning to develop an LLM-based framework for task offloading and resource allocation in digital twin-assisted

© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Mobile Computing. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TMC.2026.3745584

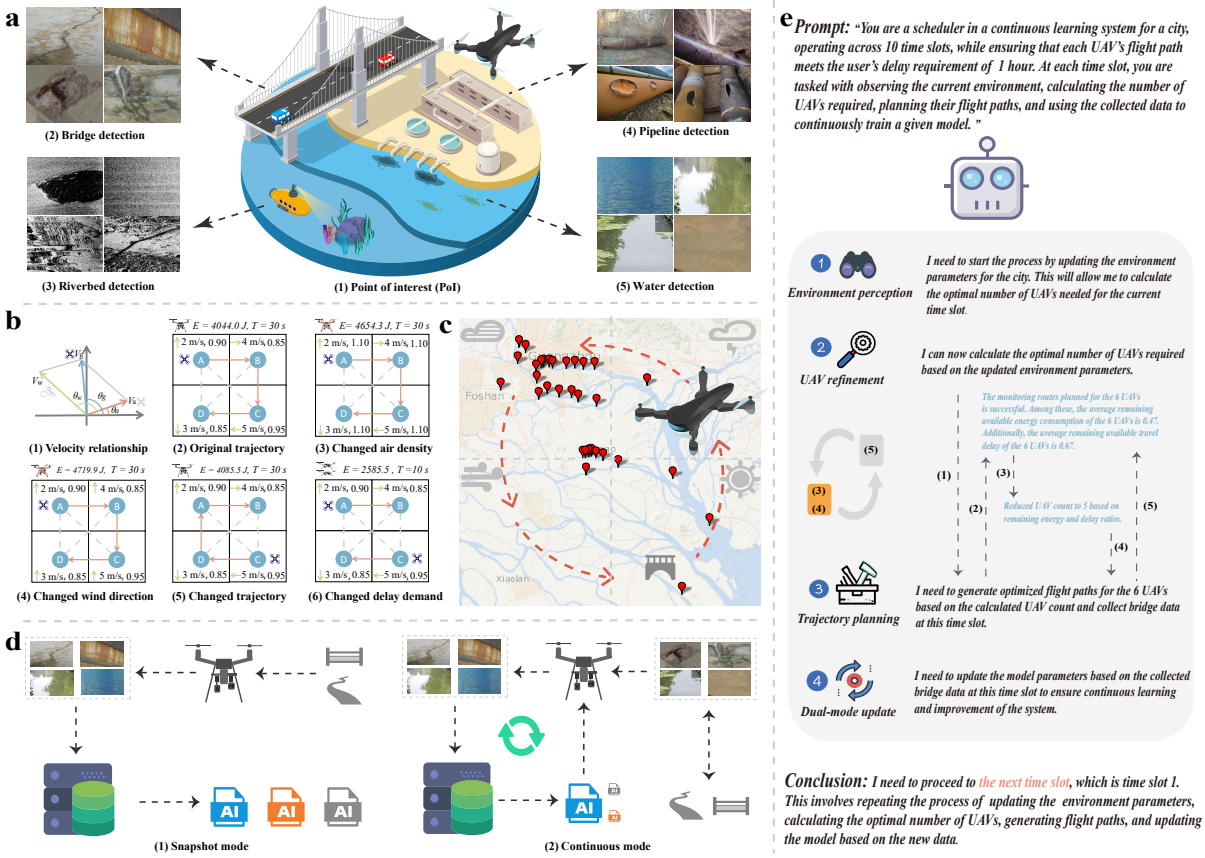


Fig. 1: An LLM agent-driven hydraulic metaverse monitoring system, where Fig. (a) showcases a variety of PoIs in hydraulic engineering, and Fig. (b) illustrates the relationships among ground speed (v_g), wind speed (v_w), and actual speed (v_a) of a UAV within a weather-affected region. Fig. (c) illustrates a UAV swarm navigating toward geographically dispersed PoIs (red marker) under dynamic weather conditions, and Fig. (d) presents the differences between the two model-update modes, where the snapshot mode uses data collected at a single point in time, while the continual mode incrementally updates the model using data collected over multiple time slots. Fig. (e) shows the execution process of the LLM agent.

edge computing networks, replacing conventional MARL for offloading decisions.

Continual learning: This technology addresses the challenge of acquiring new knowledge while retaining past learning. Lange *et al.* [8] provided a taxonomy of continual learning methods, evaluating the performance across different benchmarks and examining factors such as model capacity, regularization, and task order, along with memory consumption and computational costs. Among classic approaches, experience replay [37] combats catastrophic forgetting by storing and revisiting past data in continual training. Another foundational method, the elastic weight consolidation (EWC) algorithm [18], selectively constrains updates to critical weights, preserving learned tasks. Recently, Lopez-Paz *et al.* [24] proposed a gradient episodic memory (GEM) algorithm, which stores past task samples and enforces gradient constraints to balance knowledge retention and transfer. To further reconcile new task learning with memory stability, Li *et al.* [21] introduced the MGSER-SAM algorithm, integrating experience replay with sharpness-aware optimization. Beyond standalone models, continual learning also enhances edge-cloud systems. For example, Li *et al.* [19] leveraged this technique for digital twin synchronization, optimizing accuracy utility by developing approximation algorithms.

Hydraulic metaverse: The hydraulic metaverse is a digital twin system that integrates hydraulic infrastructure and its surrounding ecological environment. Min *et al.* [7] presented hierarchical cognitive architectures for autonomous water systems, with a particular focus on federated knowledge engines and the orchestration of domain-specific LLMs for collaborative decision-making. Li *et al.* [20] proposed a digital twin based intelligent monitoring system architecture for dam operation safety. In addition, digital twin synchronization is critical in metaverse applications. Qiu *et al.* [29] proposed a learning-based system combining proximal policy optimization with graph attention network, gated recurrent unit, and Transformer encoder for metaverse information sharing. Qiu *et al.* [30] further designed a resource scheduler with a shallow-deep distiller and an agile refresher library containing exact, approximation, and residual diffusion reinforcement learning algorithms for metaverse service refreshment.

Unlike existing works on UAV trajectory optimization that largely ignore the impact of dynamic weather conditions on energy consumption, we explicitly incorporate dynamic weather into multi-UAV trajectory optimization. The most closely related work by Sorbelli *et al.* [35] considers time-

© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Mobile Computing. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TMC.2026.3745584

varying wind but omits air density and focuses on event-driven tasks [34], whereas our work targets continual monitoring under strict delay and battery constraints. Meanwhile, hydraulic engineering studies assume static monitoring, overlooking long-term data distribution shifts and continual learning challenges such as catastrophic forgetting. Moreover, for LLM-based decision making, Tang *et al.* [36] use a lightweight LLM only as an auxiliary RL tool, whereas our system leverages an LLM as a core agent that autonomously determines UAV count, plans trajectories, and continually updates the model.

III. Preliminaries

A. Metaverse and continual monitoring requests

We model a hydraulic metaverse as a weighted graph $\mathcal{G} = (\mathcal{P} \cup \mathcal{V}, \mathcal{E})$, where $\mathcal{P}$ denotes a set of PoIs including bridges, tunnels, water quality, and other essential objects that require regular and reliable inspections. $\mathcal{V}$ denotes a set of base stations spatially distributed across a city, responsible for routing data between detection models and users. A subset $\mathcal{D} \subset \mathcal{V}$ corresponds to fixed depots co-located with selected base stations, serving as UAV launch and return points. In addition, the edge set $\mathcal{E}$ captures feasible flight links for UAVs between $\mathcal{P}$ and $\mathcal{V}$, where each edge e_l has a fixed distance attribute $\mathfrak{d}_l$ and a weather-dependent energy consumption attribute $\mathfrak{e}_l$. To model dynamic weather conditions across different regions, the city is partitioned into a two-dimensional grid, where each weather region $w_q \in \mathcal{W}$, including multiple cells, represents a distinct geographical area with local weather parameters including wind speed, wind direction, and air density. Here, wind direction is discretized into eight equally spaced intervals spanning $360°$, with east designated as the reference direction ($0°$). Note that these weather parameters are updated at fixed time intervals $t \in \mathbb{T}$. Within the established coordinate system shown in Fig. 1(b)(1), the UAV trajectory is governed by the vector relationship between the environmental and its own parameters. For instance, in weather region w_q, the actual navigation vector $\mathbf{n}_{a,k,q} = (v_{a,k,q}, \theta_{a,k,q})$ is determined by the wind vector $\mathbf{n}_w = (v_{w,k,q}, \theta_{w,k,q})$ and the required ground navigation vector $\mathbf{n}_{g,k} = (v_{g,k}, \theta_{g,k})$ of a UAV k, namely,

$$v_{a,k,q} = \sqrt{v_{g,k}^2 + v_{w,k,q}^2 - 2v_{g,k}v_{w,k,q}\cos(\Delta\theta_{k,q})}, \quad (1)$$

where $\Delta\theta_{k,q} = |\theta_{w,k,q} - \theta_{g,k}|$ denotes the absolute angular difference between wind direction and ground navigation direction, and the corresponding angle is calculated as

$$\theta_{a,k,q} = \arctan\left(\frac{v_{g,k}\sin\theta_{g,k} - v_{w,k,q}\sin\theta_{w,k,q}}{v_{g,k}\cos\theta_{g,k} - v_{w,k,q}\cos\theta_{w,k,q}}\right) + \pi \cdot \mathbb{I}_{\{v_{g,k}\cos\theta_{g,k} - v_{w,k,q}\cos\theta_{w,k,q} < 0\}} \mod 2\pi. \quad (2)$$

When a user intends to call a monitoring service, they issue a continual monitoring request from their mobile device to the nearest edge server. This request $\mathcal{R}_i = \{\mathfrak{T}_i, \mathcal{M}_i, \mathbb{T}_i\}$ is modeled as a tuple comprising the delay requirement $\mathfrak{T}_i$, the model type $\mathcal{M}_i$, and the number of monitoring slots $\mathbb{T}_i$. Upon receiving request $\mathcal{R}_i$, an edge-deployed agent autonomously dispatches UAVs to perform data collection within the hydraulic metaverse, while concurrently executing learning processes that leverage the collected data. Ultimately, the monitoring result is relayed by the base station and delivered to the mobile device of the user.

B. Metrics of UAV monitoring process

Delay: During data collection from PoIs, a UAV experiences delays due to both flight and data transmissions. The flight delay primarily depends on the distance $\mathfrak{d}_{(u,v)}$ of selected path $\mathfrak{p}_{(u,v)}$ between two PoIs p_u and p_v in $\mathcal{G}$, and the ground speed v_g of the UAV k, represented by $\mathcal{T}^f_{k,u,v} = \frac{\mathfrak{d}_{(u,v)}}{v_{g,k}}$. In the data collection stage, two scenarios are considered based on whether the PoI is equipped with a sensor, as follows.

Case 1 (The UAV collects data through camera scanning): The UAV collects data from elongated PoIs, such as roads, pipelines, or bridges, through camera-based segmented scanning. For instance, in bridge fault detection, the UAV divides the bridge into smaller segments and scans each segment sequentially, as the full structure may exceed the field of view of the camera. Let t_s represent the delay of a single scan, and let l_j denote the physical length of PoI p_j. Accordingly, the hovering delay of scanning p_j is given by

$$\mathcal{T}^s_{k,j} = \lceil \alpha \cdot l_j \rceil \cdot t_s, \quad (3)$$

where α is the scan density coefficient, reflecting the relationship between the number of scan segments and the physical length of the PoI. A higher α indicates denser scanning, which improves data quality but increases time and energy consumption.

Case 2 (The UAV collects data by communicating with ground sensors): Assuming UAV k hovers directly above a PoI p_j (the distance $\mathfrak{d}_{k,j}$ between k and p_j equals the hovering altitude H), it begins data collection before proceeding to the next point $p_{j'}$. A statistical model is adopted to characterize the air-to-ground communication channel. The hovering delay depends on both the data transmission rate and data volume of the sensor in PoI p_j. Specifically, the line-of-sight (LoS) probability between k and p_j is given by $\mathfrak{P}^{\text{LoS}}_{k,j} = \frac{1}{1+\zeta\cdot\exp[-\eta\cdot(90-\zeta)]}$, where ζ and η are environment-dependent parameters. In addition, the non-line-of-sight (NLoS) probability is $\mathfrak{P}^{\text{NLoS}}_{k,j} = 1 - \mathfrak{P}^{\text{LoS}}_{k,j}$, and the average linear-scale path-loss factor is $L_{k,j} = \hat{\alpha}\left(\mathfrak{P}^{\text{LoS}}_{k,j}\cdot\mu_{\text{LoS}} + \mathfrak{P}^{\text{NLoS}}_{k,j}\cdot\mu_{\text{NLoS}}\right)$, where $\hat{\alpha} = (\mathfrak{d}_{k,j})^{\gamma}\cdot\left(\frac{4\pi f_c}{c}\right)^2$, f_c is the carrier frequency, c is the speed of light, and γ is the path-loss exponent. Here, μ_{LoS} and μ_{NLoS} are constant additional loss factors under LoS and NLoS conditions. The received signal power and signal-to-noise ratio (SNR) at UAV k are $\mathfrak{P}^{\text{rx}}_{k,j} = \mathfrak{P}^{\text{tr}}_j / L_{k,j}$ and $\Gamma_{k,j} = \mathfrak{P}^{\text{rx}}_{k,j}/\sigma^2$, respectively [17], where $\mathfrak{P}^{\text{tr}}_j$ is the transmission power of the sensor at p_j and σ^2 is the noise power. According to the Shannon theorem, the achievable transmission rate is $R_{k,j} = B\log_2(1+\Gamma_{k,j})$, where B is the bandwidth. All in all, the hovering time of UAV k for collecting data from PoI p_j is

$$\mathcal{T}^h_{k,j} = \frac{|\mathfrak{D}_j|}{R_{k,j}}, \quad (4)$$

where $|\mathfrak{D}_j|$ is the data size from PoI p_j.

© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Mobile Computing. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TMC.2026.3745584

The monitoring delay of UAV k at PoI p_j is formulated as

$$\widehat{\mathcal{T}}_{k,j} = \widetilde{\alpha}_j \mathcal{T}^s_{k,j} + (1-\widetilde{\alpha}_j)\mathcal{T}^h_{k,j}, \tag{5}$$

where binary variable $\widetilde{\alpha}_j \in \{0,1\}$ indicates the data collection mode of PoI p_j, namely, $\widetilde{\alpha}_j = 1$ denotes camera scanning, while $\widetilde{\alpha}_j = 0$ corresponds to sensor communication. Furthermore, the total monitoring delay of a flight tour of UAV k is

$$\mathcal{T}_k = \sum_{e_{(u,v)} \in \mathcal{E}} \mathcal{T}^f_{k,u,v} \cdot r_{k,u,v} + \sum_{p_j \in \mathcal{P}} \widehat{\mathcal{T}}_{k,j} \cdot x_{k,j}, \tag{6}$$

where binary variable $r_{k,u,v} \in \{0,1\}$ indicates whether UAV k traverses an edge $e_{(u,v)} \in \mathcal{E}$, while binary variable $x_{k,j} \in \{0,1\}$ indicates whether UAV k collects data from PoI p_j. To meet the user delay requirement, UAV k must satisfy the following constraint

$$\mathcal{T}_k \le \mathfrak{T}_i. \tag{7}$$

Energy consumption: Each UAV is equipped with a battery budget $\mathfrak{B}$. Its energy consumption consists of propulsion energy during edge traversal and data-collection energy at PoIs. The latter comprises scanning and communication components. For the scanning process, let $\mathfrak{P}_s$ denote the power of a UAV during scanning targets. The total scanning energy consumed by UAV k can be expressed as

$$E^s_k = \sum_{p_j \in \mathcal{P}} \widetilde{\alpha}_j \cdot \mathfrak{P}_s \cdot \mathcal{T}^s_{k,j} \cdot x_{k,j}, \tag{8}$$

For the communication process, let $\mathfrak{P}^{\text{cir}}_k$ denote the receiver-circuit power of UAV k. Its communication energy consumption is

$$E^c_k = \sum_{p_j \in \mathcal{P}} (1-\widetilde{\alpha}_j) \cdot \mathfrak{P}^{\text{cir}}_k \cdot \mathcal{T}^h_{k,j} \cdot x_{k,j}. \tag{9}$$

In addition to data collection, propulsion is another major source of UAV energy consumption. The propulsion power $\mathfrak{P}(v_a, \rho)$ of a UAV mainly consists of three components: *blade profile*, *induced*, and *parasite* power, respectively [45], which can be modeled as

$$\mathfrak{P}(v_a,\rho) = \frac{1}{2} d_0 \rho s A v_a^3 + \mathfrak{P}_0 \left(1 + \frac{3v_a^2}{U^2_{tip}}\right) + \mathfrak{P}_1 \left(\sqrt{1 + \frac{v_a^4}{4v_0^4}} - \frac{v_a^2}{2v_0^2}\right)^{1/2}, \tag{10}$$

where v_a is the actual speed of a UAV, and the air density $\rho = \frac{\mathfrak{p}}{287.05\times(\mathfrak{t}+273.15)}$ depends on the atmospheric pressure $\mathfrak{p}$ and ambient temperature $\mathfrak{t}$ in the current environment. The first term of Eq. (10) corresponds to the *parasite* power during forward flight, and the remaining two terms represent the *blade profile* and *induced* power, respectively. For $v_a = 0$, the power function reduces to $\mathfrak{P}(0,\rho) = \mathfrak{P}_0 + \mathfrak{P}_1$, where $\mathfrak{P}_0 = \frac{\delta}{8}\rho s A \Omega^3 R^3$, $\mathfrak{P}_1 = (1+k)\frac{W^{3/2}}{\sqrt{2\rho A}}$, and $v_0 = \sqrt{\frac{W}{2\rho A}}$ with detailed parameter definitions provided in [45].

Property 1. *Given a hydraulic metaverse with dynamic weather conditions where weather region w_q is characterized by air density ρ, wind speed $v_{w,k,q}$, and wind direction $\theta_{w,k,q}$, and a UAV k with the parameters adopted from [45], when $v_a > 2v_0$, the propulsion power in Eq. (10) can be approximated as $\mathfrak{P}(v_a,\rho) \approx 0.0075 v_a^3 \rho + 65.18\rho + 0.0136 v_a^2 \rho + \frac{437.38}{v_a \rho}$. Moreover, under headwind conditions yielding $v_a \in [\overline{v}_a, v_{a_{\max}}]$, the simplified propulsion power is strictly increasing with $\rho \in [\rho_{\min}, \rho_{\max}]$ for any fixed v_a, where $\overline{v}_a = 10$ m/s, $\rho_{\min} = 0.8$, and $\rho_{\max} = 1.6$.*

Proof. The proof can be found in Appendix A. □

Throughout this paper, edge-traversal propulsion power is evaluated using the simplified expression above for $v_a > 0$, while $\mathfrak{P}(0,\rho) = \mathfrak{P}_0 + \mathfrak{P}_1$ is used when $v_a = 0$. Although low-speed segments may introduce approximation error, their impact on the total energy consumption is small.[3] The original model in Eq. (10) can directly replace the simplified expression by recalculating the edge-energy values without changing the problem formulation or the proposed algorithms.

Consistent with the implementation, propulsion energy is evaluated during edge traversal. At each PoI, the data-collection energy accounts for the scanning or receiver-circuit energy through E^s_k and E^c_k. Station-keeping propulsion during data collection is not separately included. The propulsion energy of UAV k along path $p_{(u,v)}$ between PoIs u and v is

$$E^f_{k,p_{(u,v)}} = \sum_{e_l \in p_{(u,v)}} \sum_{w_q \in \mathcal{W}} \beta_{l,q} \cdot \frac{\mathfrak{d}_l}{v_{g,k}} \cdot \mathfrak{P}(v_{a,k,q}, \rho_q), \tag{11}$$

where $\beta_{l,q}$ represents the percentage of the projected length of weather region w_q on link e_l to the length $\mathfrak{d}_l$ of the link.

The total energy consumption of UAV k during a monitoring tour is

$$E_k = \sum_{e_{(u,v)} \in \mathcal{E}} E^f_{k,p_{(u,v)}} \cdot r_{k,u,v} + E^s_k + E^c_k. \tag{12}$$

To meet its energy feasibility, UAV k must satisfy the following constraint [4],

$$E_k \le \mathfrak{B}_k. \tag{13}$$

C. Problem Definition

Definition 1. *Given a smart hydraulic metaverse network $\mathcal{G} = (\mathcal{P} \cup \mathcal{V}, \mathcal{E})$ comprising target monitoring zones in $\mathcal{P}$ and base stations in $\mathcal{V}$ serving as potential UAV depots in $\mathcal{D}$, where each edge $e_l \in \mathcal{E}$ has weather-dependent traversal energy $\mathfrak{e}_l(t)$ that varies temporally due to environmental conditions, the continual monitoring problem of a UAV swarm is to determine the smallest number κ of UAVs and κ optimal cyclic routes for the UAVs to cover all zones, with each zone being visited exactly once by one UAV, while minimizing the total energy consumption of all UAVs, where each UAV must start and*

[3] Using the original power model as the reference, re-evaluating the feasible routes from the mixed-weather scalability experiment in Fig. 18 while holding the selected routes fixed gives an average relative error of 0.20% and a maximum of 0.36% in total energy consumption.

[4] If a disruption occurs, the UAV may proceed to the next PoI or divert to the nearest depot based on its remaining battery. Upon returning with critically low battery, the collected data can be offloaded via the collocated base station, and a replacement UAV or recharging may support mission continuity. These emergency recovery mechanisms are outside the scope of this work and are not included in the planning model.

© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Mobile Computing. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TMC.2026.3745584

return to its assigned depot and multiple UAVs cannot use the same depot at any given time slot t, subject to UAV battery capacities and the user delay requirement.

Theorem 1. *The continual monitoring problem of a UAV swarm in the smart hydraulic metaverse is NP-hard.*

Proof. The proof can be found in Appendix B. □

Theorem 1 implies that obtaining globally optimal solutions is computationally expensive. In the dynamic weather-aware setting, edge-traversal energy changes over time, making a computed solution no longer optimal and thus requiring frequent replanning. In other words, the exact optimization is impractical for real-time UAV scheduling, motivating the proposed system for high-quality online decision-making.

D. ILP formulation

We formulate a single-slot integer linear program (ILP) over connections between distinct PoIs or between a PoI and a depot. In the following constraints, each connection represents a precomputed minimum-energy path under the observed weather, with its corresponding energy and delay costs. Binary variables z_k, $y_{k,d}$, $x_{k,j}$, and $r_{k,u,v}$ represent UAV activations, depot assignments, PoI assignments, and connection selections, respectively. Variable $u_{k,j}$ is an integer variable that prevents sub-tours, ensuring each UAV follows one continuous path. We here adopt a lexicographic optimization objective for the problem, that is, we minimize the number of activated UAVs, followed by minimizing the total energy consumption among all feasible solutions when the minimum number of UAVs is found. Specifically, the first stage of the problem is to find

$$\kappa = \min \quad \sum_{k\in\mathcal{K}} z_k, \tag{14}$$

subject to (7), (13)

$$\sum_{k\in\mathcal{K}} x_{k,j} = 1, \quad \forall p_j \in \mathcal{P}, \tag{15}$$

$$x_{k,j} \le z_k, \quad \forall k \in \mathcal{K},\ \forall p_j \in \mathcal{P}, \tag{16}$$

$$\sum_{k\in\mathcal{K}} y_{k,d} \le 1, \quad \forall d \in \mathcal{D}, \tag{17}$$

$$\sum_{d\in\mathcal{D}} y_{k,d} = z_k, \quad \forall k \in \mathcal{K}, \tag{18}$$

$$u_{k,i} - u_{k,j} + |\mathcal{P}|\, r_{k,i,j} \le |\mathcal{P}| - 1, \quad \forall k \in \mathcal{K},\ \forall e_{(i,j)} \in \mathcal{E} \text{ with } p_i, p_j \in \mathcal{P},\ i \ne j, \tag{19}$$

$$x_{k,j} \le u_{k,j} \le |\mathcal{P}|\, x_{k,j}, \quad \forall k \in \mathcal{K},\ \forall p_j \in \mathcal{P}, \tag{20}$$

$$\sum_{u:e_{(u,j)}\in\mathcal{E}} r_{k,u,j} = x_{k,j}, \quad \forall k \in \mathcal{K},\ \forall p_j \in \mathcal{P}, \tag{21}$$

$$\sum_{v:e_{(j,v)}\in\mathcal{E}} r_{k,j,v} = x_{k,j}, \quad \forall k \in \mathcal{K},\ \forall p_j \in \mathcal{P}, \tag{22}$$

$$\sum_{j:e_{(d,j)}\in\mathcal{E}} r_{k,d,j} = y_{k,d}, \quad \forall k \in \mathcal{K},\ \forall d \in \mathcal{D}, \tag{23}$$

$$\sum_{j:e_{(j,d)}\in\mathcal{E}} r_{k,j,d} = y_{k,d}, \quad \forall k \in \mathcal{K},\ \forall d \in \mathcal{D}, \tag{24}$$

where Eq. (7) ensures that the total monitoring delay of each UAV satisfies its user's delay requirement, while Eq. (13) guarantees that the total energy consumption of each UAV is no greater than its battery budget. Eq. (15) guarantees that each PoI is assigned to exactly one UAV only. Eq. (16) states that UAV k must be activated if it is assigned to any PoI. Eq. (17) ensures that each depot can serve at most one UAV, while Eq. (18) requires each activated UAV to be associated with exactly one depot. Eqs. (19) and (20) are the sub-tour elimination constraints, where $u_{k,j}$ denotes the visiting order of PoI p_j on the route of UAV k, thereby preventing the formation of multiple disconnected cycles. Eqs. (21) and (22) enforce flow conservation at each visited PoI, meaning that UAV k must enter and leave every assigned PoI exactly once. Eqs. (23) and (24) are the closed-loop constraints, which guarantee that each UAV departs from and returns to its assigned depot.

Having obtained the minimum number κ of UAVs, the second stage of the problem is to minimize the total energy consumption when the number of activated UAVs is fixed, namely we aim to

$$\text{minimize} \quad \sum_{k\in\mathcal{K}} E_k \tag{25}$$

$$\text{subject to} \quad \sum_{k\in\mathcal{K}} z_k = \kappa, \tag{26}$$

together with Eqs. (7), (13), (15)–(24). This lexicographic formulation aims to minimize the total energy consumption only among feasible solutions.

IV. An LLM Agent-Driven UAV Monitoring System

The LLM agent-driven UAV swarm monitoring system consists of four fundamental elements, including perception, reasoning, execution, and memory, which collectively form the intelligent control center. The agent executes tasks through an iterative observe-think-act cycle until completion. In the observation phase, the agent actively observes the environment using perception tools to gather real-time data and store the data in memory, while simultaneously retrieving relevant historical records and task specifications from memory. During its thinking phase, the agent analyzes these fresh observations and formulates an efficient action plan. In the action phase, the agent dynamically selects the most appropriate tools to execute the planned operations, while documenting the outcomes in its memory, as shown in Fig. 1(e). The pseudocode of the proposed system is presented in Algorithm 1[5].

A. Non-execution component

The non-execution components for the agent are responsible for memory, observation, and reasoning.

Memory is the core component connecting all system elements. It is divided into long-term and short-term categories. Long-term memory stores persistent information such as UAV

[5]Some implementation details are publicly available at https://github.com/yurain26/Hydraulic_Metaverses/tree/main. The trust boundaries, threat model, security mechanisms, and corresponding validation experiments are presented in Appendix C of the supplementary material.

© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Mobile Computing. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TMC.2026.3745584

Algorithm 1: UAV Monitoring Agent System

Input: Hydraulic metaverse network $\mathcal{G}$, a large language model $\mathfrak{L}$, designed prompts $\mathbb{P}$, and a detection model $\mathcal{M}$.
Output: A model that continuously learns by collecting data through the UAV swarm.

begin
  $\mathbb{P}_a \leftarrow$ "You are a scheduler in a continuous learning system for hydraulic metaverse $\mathcal{G}$ in a city, operating across $\mathbb{T}$ time slots, while ensuring that each UAV's flight path meets the user's delay requirement of $\mathfrak{T}_i$ seconds. At each time slot, you are tasked with observing the current environment, calculating the number of UAVs required, planning their flight paths, and using the collected data to continuously train a given model. Available Tools:
  - $\widetilde{\mathcal{G}} \leftarrow$ `Environment Perception`($\mathcal{G}$)
  - $\kappa \leftarrow$ `UAV Count Refinement`($\widetilde{\mathcal{G}}$, $\mathbb{P}$)
  - $\mathfrak{D} \leftarrow$ `UAV Trajectory Planning`($\widetilde{\mathcal{G}}$, κ)
  - $\widetilde{\mathcal{M}} \leftarrow$`Dual-mode Model Update`($\mathfrak{D}$, $\mathcal{M}$)";
  Call the agent using large language model $\mathcal{L}(\mathbb{P}_a)$;
  return A well-trained detection model $\widetilde{\mathcal{M}}$;

Algorithm 2: UAV Count Refinement module

Input: Hydraulic metaverse network $\mathcal{G}$, a large language model $\mathfrak{L}$, and the designed prompts $\mathbb{P}$.
Output: The selected UAV count $|\mathbb{S}|$ and trajectory plan $\mathbb{S}$, or failure

begin
  $\mathbb{P}_{c,t} \leftarrow$ "Based on the current environment and information $\mathfrak{GI}_t$ in $\mathcal{G}$, preliminarily calculate the number κ of UAVs needed" ;
  $\kappa \leftarrow$ Call the large language model $\mathfrak{L}(\mathbb{P}_{c,t}, \mathfrak{GI}_t)$;
  Initialize $\mathcal{CL} \leftarrow \emptyset$;
  for *Evolution level* $el \in \mathfrak{EL}$ **do**
    $\mathbb{S} \leftarrow$ `UAV Trajectory Planning`($\mathcal{G}$, κ);
    $\mathbb{P}_{ua} \leftarrow$ `Demand Check`($\mathbb{S}$);
    if $\mathbb{P}_{ua}$ *indicates feasibility* **then**
      $\mathcal{CL} \leftarrow \mathcal{CL} \cup \{(|\mathbb{S}|, \mathbb{S})\}$;
      $\mathbb{P}_{r,t} \leftarrow$ "Reduce the number of UAVs based on the current available delay and energy information in $\mathbb{P}_{ua}$ of the UAV swarm";
    else
      $\mathbb{P}_{r,t} \leftarrow$ "Add the number of UAVs based on the current excessive delay and energy information in $\mathbb{P}_{ua}$ of the UAV swarm";
      if el *is not the final level* **then** $\kappa \leftarrow \mathfrak{L}(\mathbb{P}_{r,t})$;
  if $\mathcal{CL} = \emptyset$ **then return** failure;
  return The UAV count and trajectory plan selected from $\mathcal{CL}$ by minimizing UAV count first and total energy second;

Function `Demand Check`($\mathbb{S}$)**:**
  if $\mathbb{S}$ *is feasible* **then**
    return True, and a normalized available delay as well as the amount of residual energy;
  else
    return False, and normalized violation values on delay and energy consumption;

parameters, user requirements, mission prompts, and task status. Short-term memory manages dynamic data, including real-time environmental states and post-action updates, which is continually refreshed during the agent execution.

Perception includes inward and outward processes. Inward perception enables the agent to reflect on its operational status by retrieving and analyzing the stored memory through similarity-based retrieval. Outward perception allows the agent to observe external environments using perception tools, such as web APIs, to acquire real or simulated data (e.g., weather conditions).

Reasoning relies on an LLM to make decisions through structured prompts and observational data. At initialization, a setup prompt defines objectives, metrics, and constraints. During its execution, the agent iteratively refines its action plan by integrating real-time environmental inputs and memory data, ensuring alignment between strategies and their execution. Finally, the agent selects the most suitable tool by evaluating its functional capability against current objectives and available evidence.

B. Execution component

Execution bridges reasoning and action by converting decisions into sub-tasks using specialized tools, while providing bidirectional feedback on outcomes and failures. For continual monitoring missions, we designed a sequential toolkit. Note that the environment perception tool was introduced in the perception component.

1) The self-evolving swarm scheduling tool: The self-evolution of this tool is achieved through a feedback-driven refinement mechanism in the UAV count refinement module and a bottom-up UAV merging strategy in the UAV swarm trajectory planning module. The complete pseudocode for the aforementioned modules is provided in Algorithms 2 and 3.

UAV count refinement module: This module searches for a feasible number of UAVs required to cover all PoIs while satisfying the delay, energy consumption, and fixed depot constraints. Rather than relying on static numerical optimization, the module introduces a refinement process guided by evolution levels. Specifically, it consists of the following steps. **Step 1:** The agent makes a coarse estimation of the UAV count based on the current observations $\mathfrak{GI}_t = \{\mathcal{V}, \mathbf{n}_w, \rho, v_{g,k}, \mathfrak{B}_k, \mathfrak{T}_i\}$, including spatial distributions of nodes in $\mathcal{G}$, environmental conditions at slot t (wind vector $\mathbf{n}_w$ and air density ρ), UAV parameters (ground speed $v_{g,k}$ and battery energy level $\mathfrak{B}_k$), and user delay requirement $\mathfrak{T}_i$. **Step 2:** The module enters a multi-round refinement process guided by the evolution level [6]. At each level, the module generates UAV flight trajectories using the trajectory planning module, evaluates the generated plan feasibility against delay and energy constraints for each UAV, and provides feedback to the agent. This feedback-driven adjustment constitutes one evolutionary iteration that enables the agent to progressively converge toward an improved allocation. When all constraints are satisfied, the module is able to analyze the surplus energy and delay margins to reduce the UAV count; conversely, when

[6]The evolution level counts trajectory-planning evaluations, including the initial estimate. The default is five evaluations (one initial evaluation and four refinements) unless otherwise specified.

© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Mobile Computing. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TMC.2026.3745584

Algorithm 3: UAV Trajectory Planning Module

Input: Network $\mathcal{G}$ and estimated UAV count κ.
Output: Trajectory plan $\mathbb{S}$ with per-UAV detection data $\mathfrak{D}_k$ (empty before execution).

begin
  Partition the set of PoIs $\mathcal{P}$ into κ clusters $\mathcal{TC}$, using the K-means algorithm;
  For each cluster $tc_k \in \mathcal{TC}$ and each depot $v_j \in \mathcal{D}$, calculate $(\tau_k, \widetilde{\mathcal{EC}}_{k,j}) \leftarrow \texttt{NN_2OPT}(v_j, tc_k)$ and put the energy value into the trajectory consumption matrix $\widetilde{\mathcal{EC}}$;
  Based on $\widetilde{\mathcal{EC}}$, the solution $\mathbb{S}$ that minimizes total energy consumption $\widetilde{\mathcal{EC}}_t$ is obtained using the Hungarian algorithm by assigning each cluster to a different depot;
  if $\mathbb{S}$ *is not feasible* **then**
    Run a top-down conflict avoidance mechanism;
  if $\mathbb{S}$ *is feasible* **then** Run a bottom-up UAV merging mechanism;
  foreach *cluster* tc_k *with assigned depot* $\mathcal{CD}_k$ *in* $\mathbb{S}$ **do**
    Compute $(\tau_k, \mathcal{E}_k) \leftarrow \texttt{NN_2OPT}(\mathcal{CD}_k, tc_k)$;
    Calculate total energy consumption $\mathcal{E}_k$ and delay $\mathcal{T}_k$;
    Collect detection data $\mathfrak{D}_k$ along τ_k only when $\mathbb{S}$ is selected for execution;
  return *Plan and data* $\mathbb{S} = \{(\mathcal{CD}_k, \tau_k, \mathcal{E}_k, \mathcal{T}_k, \mathfrak{D}_k)\}$;
Function NN_2OPT(*depot, pois*)**:**
  if $|pois| \leq \mathcal{D}_{thr}$ **then**
    Solve the minimum-energy tour problem using dynamic programming;
  else
    Initialize a flight trajectory τ_k for UAV k starting from *depot* that will visit the nearest unvisited PoIs until all PoIs are covered, then returns to *depot*;
    Compute total energy consumption $\mathcal{E}_k$ of the initial path;
    repeat
      for $i \leftarrow 1$ **to** $|\tau_k| - 3$ **do**
        for $j \leftarrow i + 2$ **to** $|\tau_k| - 1$ **do**
          If reversing $\tau_k[i..j]$ reduces energy, apply the change and update $\mathcal{E}_k$;
    until *there is no further improvement*;
  return τ_k, $\mathcal{E}_k$;

constraints are violated, it adaptively increases the UAV count proportional to the violation severity. **Step 3:** By exploring candidate solutions across multiple evolutionary levels, we select the feasible configuration with the smallest UAV count. If several plans use this minimum number, we choose the one with the lowest total energy consumption. The selected trajectories are retained and executed for data collection.

UAV Swarm Trajectory Planning module: This module is a key component of the self-evolving swarm scheduling tool, which optimizes UAV flight paths to further reduce energy consumption and ensure that all constraints are met. **Step 1:** The module starts by clustering PoIs into κ groups using the K-means algorithm based on the locations of PoIs, and then the Hungarian algorithm is employed to minimize the total energy consumption of all UAV-cluster pairings while ensuring distinct depots for different UAVs. **Step 2:** Although this approach offers a fast solution, it may violate energy and delay constraints on some UAVs. To address this limitation, we propose **a dual-layer optimization framework** that combines: (1) a top-down conflict avoidance mechanism that dynamically identifies the most constrained UAV and reassigns its PoIs to the nearest compliant UAVs, alleviating critical violations, and (2) a bottom-up UAV merging mechanism that consolidates the tasks assigned to two UAVs with fewer PoIs into a single UAV, while satisfying the energy and delay constraints. Once all conflicts are resolved, feasible UAV merges are performed.

In addition, an efficient function NN_2OPT is designed throughout the entire module to calculate the energy consumption $\widetilde{\mathcal{EC}}_{k,j}$ of UAV k with the tour involving candidate depot $\mathcal{CD}_k$ and a PoI cluster tc_k. In this function, we employ an adaptive optimization strategy that dynamically selects an approach based on the number of PoIs in a trajectory to strive for a fine balance between the solution quality and the computational efficiency. For PoIs below a predefined threshold, the optimal routes of UAVs can be found by dynamic programming, which provides essential information for the bottom-up UAV merging mechanism. Through precise energy calculations on small clusters, the number of UAVs can be further optimized. Specifically, these optimal trajectories help identify cases where PoIs that are mistakenly separated by K-means clustering can be efficiently served by a single UAV. For the number of PoIs exceeding the given threshold, a heuristic initializes UAV trajectories through nearest-neighbor construction, followed by local search refinement through route-segment reversal. This heuristic can significantly accelerate the calculation of energy consumption of UAVs in both the Hungarian algorithm and the top-down conflict avoidance mechanism, thus making computational efficiency more valuable than finding an optimal solution.

2) The dual-mode model update tool: The model update mode is user-specified, with continual learning as the default. Snapshot mode suits stable or single-batch scenarios, whereas continual mode handles evolving or incremental data.

Snapshot mode: We present an orthogonal detection model that uses orthogonal feature extraction and fusion for comprehensive analysis of monitoring pictures, along with a novel loss function for training stability under data imbalance. **Step 1:** The model performs a picture analysis on complementary horizontal and vertical dimensions. Horizontally, the orthogonal detection model uses traditional convolutions to extract low-, medium-, and high-frequency information across different spatial scales. Vertically, the model employs dynamic snake convolution (DSConv) [28] with deformable kernels and learned offsets to capture vertical structural features through adjustable sampling grids. **Step 2:** Through a dual-attention fusion module in the model, the orthogonal features are integrated. Specifically, channel-wise attention weighs different feature channels based on their importance, while spatial attention identifies the most relevant spatial locations. A simplified spatial pyramid pooling (SPP) module captures multi-scale contextual information, enabling the model to handle objects of varying sizes. **Step 3:** To address imbalanced data distributions in real-world datasets, we define the current-task classification loss $\mathcal{L}_c$ as a triple-guarantee loss that combines standard binary cross-entropy, positive-class-weighted binary cross-entropy, and focal weighting. For class j, the

© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Mobile Computing. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TMC.2026.3745584

Algorithm 4: Continual Model Update Tool (TRI)

Input: Collected detection data set $\mathfrak{D}_t$ and the trained detection model $\mathcal{M}_t$ at time slot t, previous task data $\mathfrak{D}_{t-1}$.
Output: A further trained model $\mathcal{M}_{t+1}$

begin
  if $t > 0$ **then**
    Compute diagonal Fisher information $\mathbf{F}_{t-1}$ and store model parameters $\boldsymbol{\theta}_{t-1}$;
  Based on the task index t, progressively freeze selected layers of $\mathcal{M}_t$;
  for *epoch* $= 1$ *to num_epochs* **do**
    for *each batch* $(x, y) \in \mathfrak{D}_t$ **do**
      Compute current-task classification loss $\mathcal{L}_c$ using Eq. (27);
      if $t = 0$ **then**
        Perform gradient descent for $\mathcal{M}_t$ by $\mathcal{L}_c$;
      else
        Compute EWC loss $\mathcal{L}_e \leftarrow \sum_{i=\max\{0,t-3\}}^{t-1} \lambda_i \sum_j \mathbf{F}_{i,j}(\theta_j - \theta_{i,j})^2$;
        Apply ASAM parameter perturbation using $\nabla(\mathcal{L}_c + \mathcal{L}_e)$ to obtain $\boldsymbol{\theta}_p$;
        Sample historical data from the memory buffer, and compute replay loss $\mathcal{L}_m$ from the sampled historical data. Recompute $\mathcal{L}_c$ and $\mathcal{L}_e$ at $\boldsymbol{\theta}_p$;
        Compute perturbed total loss $\mathcal{L}_p \leftarrow \mathcal{L}_c + \mathcal{L}_e + \alpha_l \mathcal{L}_m$ and its gradient;
        Remove the ASAM perturbation;
        Update $\mathcal{M}_t$ using the computed gradient;
      if *epoch* $=$ *num_epochs* **then**
        Store data in memory buffer;
  return $\mathcal{M}_{t+1} \leftarrow \mathcal{M}_t$

positive-sample weight is $w_j = \text{clamp}\left(\frac{N_j^{neg}+\epsilon}{N_j^{pos}+\epsilon}, 0.5, 2.0\right)$, where ϵ is a small constant for numerical stability. Since this weight is applied only to positive labels, we define $\widetilde{w}_{ij} = y_{ij} w_j + (1 - y_{ij})$. Let z_{ij} be the model logit and $p_{ij} = y_{ij}\sigma(z_{ij}) + (1 - y_{ij})(1 - \sigma(z_{ij}))$ be the confidence assigned to the correct label. The implemented weighted sum of these three classification components can then be equivalently expressed as

$$\mathcal{L}_c = -\frac{1}{N} \sum_{i,j} [\alpha_b + \alpha_c \widetilde{w}_{ij} + \alpha_f (1 - p_{ij})^{\gamma_f}] \log p_{ij}, \quad (27)$$

where N is the total number of sample-class predictions in a mini-batch, N_j^{neg} and N_j^{pos} are the numbers of negative and positive samples for class j in that mini-batch, and α_b, α_c, α_f, and γ_f denote the base, positive-class-weighting, hard-sample, and focusing coefficients, respectively.

Continual mode: To balance the adoption of new data with the preservation of prior knowledge at each time slot, we propose `TRI`, an algorithm that combines adaptive sharpness-aware minimization (ASAM) and elastic weight consolidation (EWC) with a memory replay mechanism, resulting in the progressive enhancement of the designed orthogonal model. The complete pseudocode of algorithm `TRI` is presented in Algorithm 4.

Step 1: During the initialization phase ($t = 0$), the algorithm computes the current-task classification loss $\mathcal{L}_c$ using Eq. (27) based on the current data, and performs gradient descent accordingly. **Step 2:** In the continual learning phase ($t > 0$), the algorithm adopts an integrated approach combining knowledge preservation and gradient generalization. Knowledge preservation operates at two levels. (1) At the parameter level (EWC mechanism), the diagonal Fisher information $\mathbf{F}_{t-1}$ quantifies the importance of each parameter based on previously seen data, and the parameter snapshot $\boldsymbol{\theta}_{t-1}$ stores their corresponding optimal values. These two components are jointly utilized by the EWC regularization term $\mathcal{L}_e$, which mitigates catastrophic forgetting by penalizing deviations from the stored optima of up to the three most recent tasks, with the penalty scaled according to the Fisher importance. In this term, i indexes previous tasks and j indexes model parameters. (2) At the data level (memory replay mechanism), historical samples are stored in a finite memory buffer, from which a small subset is sampled for replay learning. The memory replay loss $\mathcal{L}_m$, computed on these sampled data, helps preserve performance on previously encountered data. **Step 3:** To enhance model generalization, the algorithm incorporates both progressive freezing and perturbation-based optimization (ASAM mechanism). Progressive freezing follows the task index t, with no freezing at $t = 0$, the first two backbone modules frozen at $t = 1$, and the first four frozen at $t \geq 2$. ASAM computes parameter-scaled perturbations with radius ρ_l using $\nabla(\mathcal{L}_c + \mathcal{L}_e)$. At the perturbed parameters, $\mathcal{L}_c$ and $\mathcal{L}_e$ are recomputed, while $\mathcal{L}_m$ is computed using the sampled historical data. These losses form the perturbed total loss $\mathcal{L}_p = \mathcal{L}_c + \mathcal{L}_e + \alpha_l \mathcal{L}_m$, where $\alpha_l = \min\{0.5, 0.1 + 0.4e/E\}$ and e and E denote the current epoch and the total number of epochs, respectively. Its gradient is computed before restoring the original parameters and is then used to update the model.

All in all, catastrophic forgetting is mitigated jointly through parameter-space regularization and sample-space replay, with ASAM further smoothing the performance of the orthogonal detection model across slots.

V. Performance Evaluation

A. Experimental environment settings

We conducted experiments in a hydraulic metaverse simulated over the geographical area of Guangzhou, China, covering the area between longitudes 113.1°–113.7° and latitudes 22.6°–23.3°. The city is divided into uniform $0.1° \times 0.1°$ grids, with 42 base stations placed at grid intersections to serve as UAV depots (blue dots in Fig. 5). A dynamic weather system is also implemented with six types, including *normal* (0, 1.225), *calm* (0–10, 1.2–1.6), *breeze* (10–20, 1.0–1.2), *windy* (20–30, 0.9–1.0), *storm* (30–40, 0.85–0.9), and *hurricane* (40–50, 0.8–0.85), parameterized by wind speed (m/s) and air density (kg/m^3) [15], and wind direction following eight compass points. The weather statistics are derived from reference [15], which documents yearly weather changes across 10 stations

© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Mobile Computing. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TMC.2026.3745584

in the USA[7]. At each time slot, eight base stations were randomly selected as the centers of eight non-overlapping adverse-weather regions. Each region was assigned one of the five non-normal weather types, while all other areas remained normal.

PoIs are aligned with 32 bridges (red dots in Fig. 5) in Guangzhou, each associated with multimodal data encompassing water quality, as well as bridge, pipeline, and riverbed conditions. The monitoring horizon spans 10 time slots, and UAVs are required to complete data collection from all PoIs within 3,600 s per slot to satisfy user delay constraints [8]. To simulate data acquisition, we partition training datasets into 32 subsets, including bridge defect classification [25], riverbed classification [10], water quality classification, and pipeline defect classification (the latter two proprietary), and further divide each subset into 10 temporal segments to reflect the time-series nature of data collection. The trajectory-planning experiments use camera-based scanning at every PoI ($\widetilde{\alpha}_j = 1$), so the sensor-communication branch of the general model is not activated. UAVs follow realistic specifications with a ground speed of 20 m/s, battery capacity of 600 kJ, and scanning power consumption of 4.5 W [31], while other hardware parameters are configured according to [45]. These serve as the default settings in all experiments unless otherwise specified.

We adopted GPT-3.5-turbo as the base LLM with direct API calls unless otherwise specified. The controller comparisons in Figs. 17 and 18 used GPT-4o with a temperature of 0. In addition, the orthogonal detection model adopts a pretrained ConvNeXt-Tiny as the backbone, transforming a $3 \times 256 \times 256$ input into a feature map of size $768 \times 8 \times 8$. This feature map is then fed into a multi-scale fusion module for orthogonal feature extraction. Specifically, three parallel convolutional branches with kernel sizes 3×3, 5×5, and 7×7 (each outputting 256 channels) extract horizontal multi-scale information, while a DSConv branch (outputting 768 channels) captures vertical structural features. These orthogonal features are concatenated into $1536 \times 8 \times 8$ and compressed back to $768\times8\times8$ via channel compression. To enhance the model's ability to perceive objects of varying sizes, a simplified SPP module is applied. It performs max pooling with three scales (5×5, 9×9, 13×13), concatenates the pooled results with the original feature map ($3072 \times 8 \times 8$), and compresses the channels back to $768 \times 8 \times 8$. Finally, global average pooling flattens the $768 \times 8 \times 8$ feature map to a 768-dimensional vector, which is passed through two fully connected layers (768×256 and $256 \times$ num_classes) to produce the final logits.

[7]In simulations, the Guangzhou topology is used because the real bridge coordinates in the city are available. However, detailed weather data for Guangzhou, including temperature, humidity, wind direction, and air density, were not publicly available. We thus adopted a weather dataset from the USA.

[8]Note that shortening the time slot interval (default 3,600 s) accommodates sudden changes in learning demands, PoI status, or unexpected updates, while launching additional threads enables parallel UAV path planning for robustness against abrupt priority changes.

B. Comparison Baselines

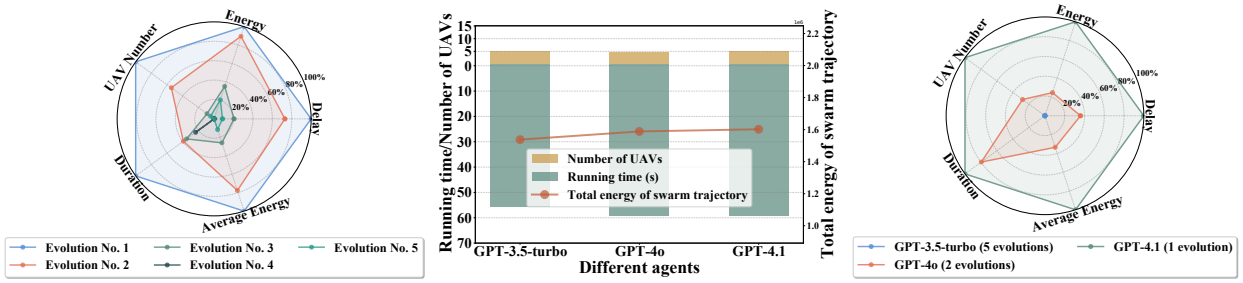
Self-evolving swarm scheduling tool. It consists of the UAV Count Refinement Module and the UAV Trajectory Planning Module. (1) For UAV count refinement, GPT-3.5-turbo, GPT-4o, and GPT-4.1 are compared under identical mission settings to investigate the impact of LLM reasoning capability on UAV swarm-size optimization. (2) For UAV trajectory planning, the optimization-based and non-LLM heuristic algorithms are coupled with the UAV Count Refinement Module: the LLM first estimates the number of UAVs, and then each algorithm generates the corresponding routes. `APP` [23] clusters PoIs among UAVs, assigns each cluster to a suitable depot, and constructs approximate closed tours using the Christofides algorithm for TSP. `TS` [14] constructs a global TSP tour, partitions the tour into energy- and delay-feasible segments, and assigns the segments to appropriate depots. `SA` [16] improves routes by relocating or exchanging PoIs and occasionally accepts suboptimal solutions to escape local optima. `PSO` [41] jointly searches for PoI assignments and visiting orders using multiple candidate solutions. In contrast, the RL-based algorithm determines both the number of UAVs and UAV routes without LLM assistance. `SAG` [38], [26] uses one centralized RL agent to determine the number of UAVs and to construct the routes for UAVs one by one, by selecting depots and unvisited PoIs. `MAG` [13], [2] models each UAV as an RL agent and jointly determines the number of UAVs and their routes through turn-based PoI selection, where additional UAVs are activated when existing agents cannot cover the rest of PoIs.

Dual-mode model update tool. It comprises snapshot-mode and continual-mode learning. (1) For snapshot-mode learning, T-CNN, VGG-A, and MetaQNN-3 [25] represent conventional CNN classifiers and an automatically searched architecture, respectively. EfficientNetV2 and EfficientNetV2+ [6] are efficient CNN backbones, while YOLOv10 [1] is a one-stage detection model. (2) For the Continual Model Update Tool, VANILLA sequentially fine-tunes the model using only new data. EWC [18] penalizes changes to parameters important for previous tasks, GEM [24] constrains harmful gradient updates using episodic memory, and REPLAY [37] jointly trains on historical and new samples. MGSER-SAM [21] combines memory-guided learning with sharpness-aware optimization.

All baseline configurations were kept identical. Unless otherwise specified, each test was run for 10 time slots without a fixed random seed. `APP` used K-means initialization with a maximum of 10 local-search iterations, while `TS` used a time limit of 120 s. For `SA`, the initial temperature, cooling rate, and maximum number of iterations were set to 150, 0.97, and 400, respectively. For the learning-based baselines, `SAG` and `MAG` represent the single-agent and multi-agent approaches, respectively. `SAG` employed an Actor–Critic network framework, while `MAG` adopted the PPO strategy. Both were trained for 1,000 episodes with an RL discount factor of 0.98, a policy learning rate of 3×10^{-4}, and a critic learning rate of 10^{-3}. For `MAG`, the PPO clip ratio and batch size were set to 0.2 and 64, respectively. For the triple-guarantee classification

© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Mobile Computing. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TMC.2026.3745584

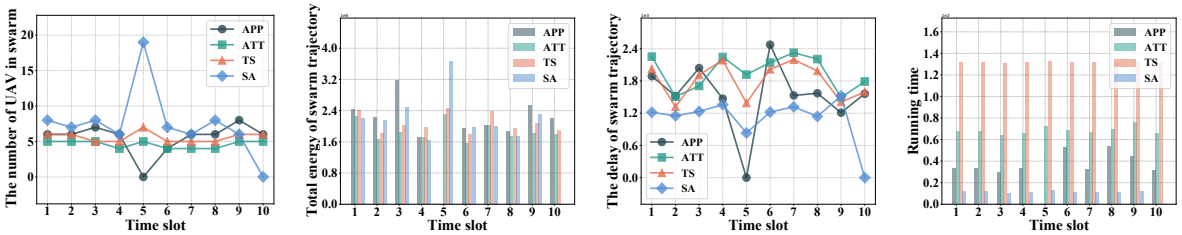


(a) Comparison of self-evolution performance among individual agent

(b) Performance comparisons of different agents with 3 evolutions

(c) Comparison of agents during self-evolution with different levels

Fig. 2: Comprehensive analysis of the agent behaviors during self-evolution.



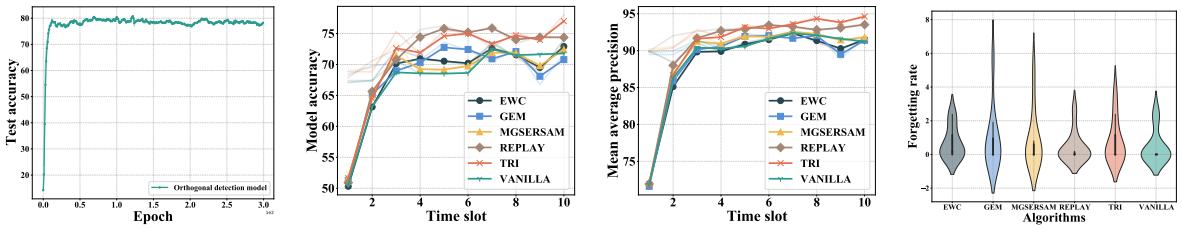
(a) The number of UAVs in the swarm

(b) The total energy consumption

(c) The average trajectory delay

(d) The running time

Fig. 3: The performance of different schedulers by varying the time slot from 1 to 10.



(a) Accuracy under snapshot mode

(b) Accuracy under continual mode

(c) The mean average precision

(d) The forgetting rate

Fig. 4: The model performance under snapshot and continual learning.

loss $\mathcal{L}_c$, the standard BCE, positive-class-weighted BCE, and focal terms were weighted by 0.3, 0.4, and 0.3, respectively. The focal term used an internal scaling factor of 0.25 and a focusing exponent of 2, while class-frequency estimation used $\epsilon = 10^{-8}$, corresponding to $\alpha_b = 0.3$, $\alpha_c = 0.4$, $\alpha_f = 0.075$, and $\gamma_f = 2$ in Eq. (27). In addition, for continual-mode learning, the learning rate and batch size were set to 6×10^{-5} and 16, respectively, with 30 training epochs per time slot. The replay memory contained up to 10,000 samples, with up to 128 historical samples drawn for each replay update. The EWC coefficient was set to 10^3, while ASAM parameters ρ_l and η were set to 0.02 and 0.01. Progressive freezing was applied to preserve stable feature layers during subsequent updates.

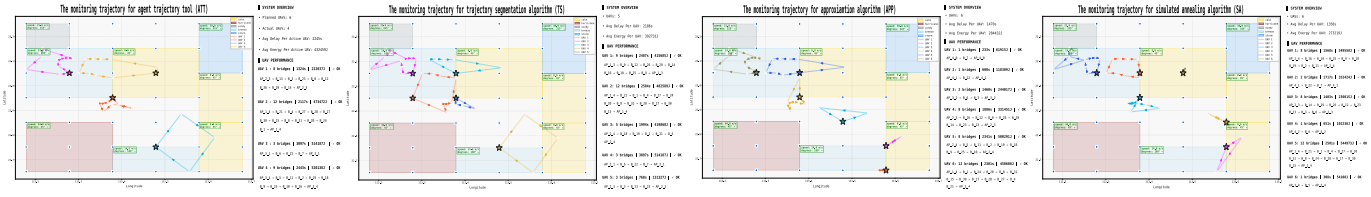
C. Comprehensive experiments for system

Self-evolution: We investigated the self-evolution performance of the agent by testing different evolution levels and base LLM models. As shown in Fig. 2(a), simulation results show that the algorithm performance improves steadily until evolution level 4 across most metrics (except duration), but fluctuates at level 5, suggesting that further evolution brings diminishing returns due to environmental dynamics and that the `GPT-3.5-turbo` agent reaches its limit. When upgrading the underlying LLM (comparing `GPT-3.5-turbo`, `GPT-4o`, and `GPT-4.1`) at a fixed evolution level 3, we observed no consistent improvement as shown in Fig. 2(b): The number of UAVs required remained the same, and total energy consumption even increased in some cases. More importantly, enhancing evolution levels contributed more significantly to agent performance than simply using a more advanced LLM. As shown in Fig. 2(c), `GPT-3.5-turbo` with high evolution outperformed `GPT-4.1` with low evolution, indicating that in dynamic environments, evolutionary adaptability is more effective than model upgrades alone.

Automation: We evaluated the automation capability of the agent through continual monitoring over 10 time slots under dynamic environmental conditions. As shown in Fig. 3, our tool, `ATT`, consistently outperformed other algorithms and resisted performance degradation, even as all methods exhibited fluctuations over time. A performance value of 0 indicates failure to plan a valid trajectory under the weather conditions in the corresponding slot. Notably, `ATT` required the fewest UAVs in all time slots, reflecting a higher UAV utilization rate. This advantage, however, involves a trade-off:

© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Mobile Computing. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TMC.2026.3745584



(a) Trajectory tool applied in agent (b) Trajectory segmentation algorithm (c) Approximation algorithm (d) Simulated annealing algorithm

Fig. 5: The monitoring trajectories of the UAV swarm delivered by different algorithms.

Dataset	Model / Accuracy (%)					
Bridge defect classification [25]	AlexNet [25]	T-CNN [25]	VGG-A [25]	DenseNet121 [25]	ENAS-3 [25]	
	66.98	67.93	70.45	70.77	72.19	
	MetaQNN-3 [25]	YOLOv10 [1]	EfficientNet-V2 [6]	EfficientNet-V2+ [6]	–	**Ours**
	72.19	74.99	77.50	79.91	–	**81.01**
Riverbed classification [10]	VGG-16 [39]	AlexNet [39]	GoogleNet [39]	CNN [11]	CNN_TL [10]	**Ours**
	79.17	81.25	88.19	90.00	92.00	**97.89**
Water quality classification	YOLO11x [4]	AlexNet	VGG-16	DenseNet121	EfficientNetV2	**Ours**
	71.62	85.41	87.30	87.84	89.46	**92.16**
Pipeline defect classification	YOLO11x [4]	AlexNet	VGG-16	DenseNet121	EfficientNetV2	**Ours**
	59.09	60.61	62.12	71.97	75.00	**77.27**

TABLE I: Comparative analysis of the accuracy of various learning-based models across datasets in snapshot mode.

Dataset	Metric	VANILLA	EWC [18]	GEM [24]	MGSER-SAM [21]	REPLAY [37]	**TRI (Ours)**
Bridge defect classification [25]	Accuracy (%)	71.84	74.05	71.68	73.26	74.37	**78.01**
	mAP (%)	91.15	91.78	91.88	91.90	93.63	**94.81**
	Forgetting rate (%)	0.86	0.51	0.81	0.78	0.78	**0.22**
Riverbed classification [10]	Accuracy (%)	94.74	95.26	94.21	95.26	94.74	**95.79**
	mAP (%)	99.22	99.42	99.43	99.47	99.44	**99.56**
	Forgetting rate (%)	0.68	0.74	1.11	0.58	**0.37**	0.53
Water quality classification	Accuracy (%)	86.22	87.30	85.68	82.43	85.14	**88.38**
	mAP (%)	94.06	93.94	93.53	92.75	92.59	**94.34**
	Forgetting rate (%)	0.46	0.70	0.89	0.85	**0.41**	0.78
Pipeline defect classification	Accuracy (%)	65.91	70.45	70.45	68.94	65.91	**73.48**
	mAP (%)	58.67	62.14	65.78	68.34	70.89	**74.63**
	Forgetting rate (%)	1.36	2.42	0.53	1.59	0.76	**0.15**

TABLE II: Comparative analysis of continual learning methods across datasets and metrics.

Maintaining full PoI coverage may lead to slightly increased energy consumption and delay. For example, when the number of UAVs is relatively close, such as in time slot 3, ATT outperforms TS in terms of both energy consumption and delay. Similarly, in time slot 9, ATT achieves lower energy consumption than APP, while maintaining a comparable delay.

Persistence: We evaluated the agent's persistence in both snapshot and continual learning modes. In snapshot mode, data is collected in a single round for orthogonal detection models; in continual mode, data is incrementally gathered over time for the TRI algorithm.

(1) **Snapshot mode**. We investigated the accuracy of the orthogonal detection model across four datasets by varying the number of training epochs from 0 to 300. The experiments were conducted with a batch size of 64 and a learning rate of 1×10^{-5}, using ConvNeXt-Tiny as the backbone model. Fig. 4(a) demonstrates that the model achieves relatively high accuracy within the first 50 epochs, followed by steady convergence and incremental improvements for the bridge defect classification scenario. Furthermore, as shown in Tab. I, our proposed model outperforms state-of-the-art methods across various scenarios, achieving an accuracy improvement of up to 5.89 percentage points.

(2) **Continual mode**. We compared TRI against other continual learning methods over multiple time slots. Each slot included 30 training epochs with a learning rate of 6×10^{-5}, a batch size of 16, and a memory sampling size of 128. For bridge defect classification, as shown in Fig. 4(b), TRI consistently outperforms other algorithms in terms of model accuracy over time, achieving up to 78% accuracy, while others do not exceed 75%. In Fig. 4(c), the mean average precision (mAP) of TRI approaches 95%, further confirming its efficiency. Fig. 4(d) shows that the Replay method, which retrains on the entire memory buffer, results in the shortest and thinnest non-zero section in its violin plot, indicating a relatively low degree of forgetting. The Vanilla method, which only trains on the current data without any continual learning mechanism, displays varying accuracy across different levels of forgetting, highlighting its instability. TRI effectively mitigates forgetting and eliminates the need to relearn all memories. Although it inevitably experiences some forgetting (e.g., achieving an accuracy of 75% as early as time slot 2, significantly higher than other algorithms), this does not compromise its final performance. In addition, as shown in Tab. II, our proposed model outperforms state-of-the-art methods across various scenarios in terms of accuracy and mAP. However, its forgetting rate is occasionally higher than that of Replay because TRI trains on a limited sample of historical data rather than the entire memory buffer.

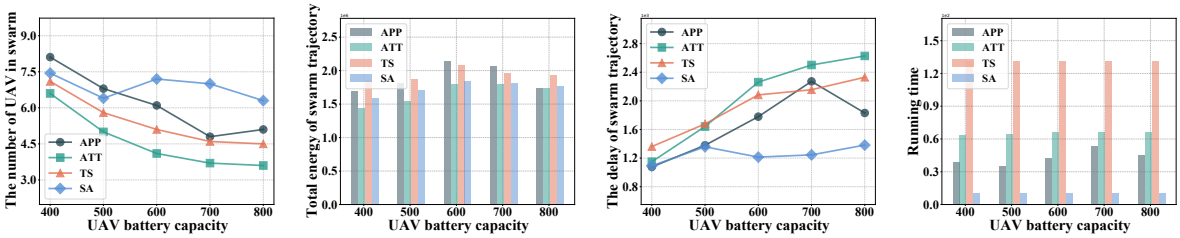
End-to-end: We studied the end-to-end performance of the agent by visualizing trajectory schedulers in a specific time slot in Fig. 5. Results show that energy consumption and swarm size are jointly affected by meteorological and geographical factors. Our tool, ATT, effectively handles this coupling effect. For example, in Fig. 5(a), ATT plans energy-efficient paths for UAV 1 and UAV 4 under adverse weather, while TS requires three UAVs for the same task. Even with the same PoIs assigned to a single UAV (e.g., UAV 2 in Fig. 5(a) and UAV 5 in Fig. 5(d)), ATT consumes less energy than SA. Moreover, by pre-clustering geographically proximate PoIs, as shown in Fig. 5(a) and (c), ATT avoids long distances, further saving energy compared to trajectories like UAV 4 in APP.

D. Non-environmental parameters for system

UAV battery capacity: Battery capacity critically shapes UAV swarm scheduling by trading off UAV count against per-UAV workload. To validate this, we varied battery capacity $\mathfrak{B}$ from 400 to 800 kJ. As shown in Fig. 6(a), larger capacities allow ATT to cut UAV usage by 0.8%–89.1%, greatly improving utilization compared with other algorithms, but at the cost of higher energy consumption and longer delays per UAV. When the number of UAVs becomes comparable across algorithms, ATT shows clear advantages. For example, at 400 kJ, it achieves 28.0% lower energy and 18.4% lower

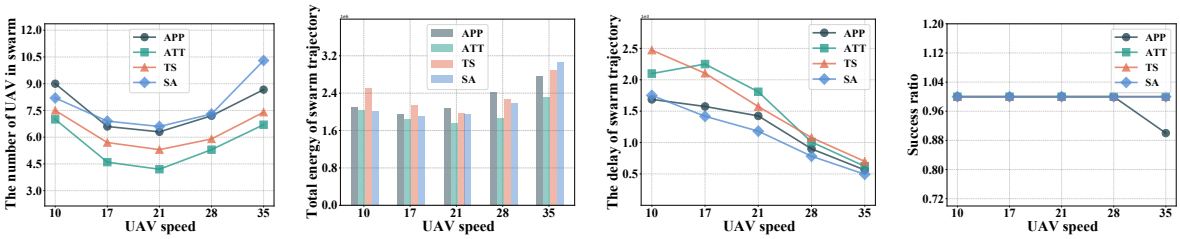
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Mobile Computing. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TMC.2026.3745584



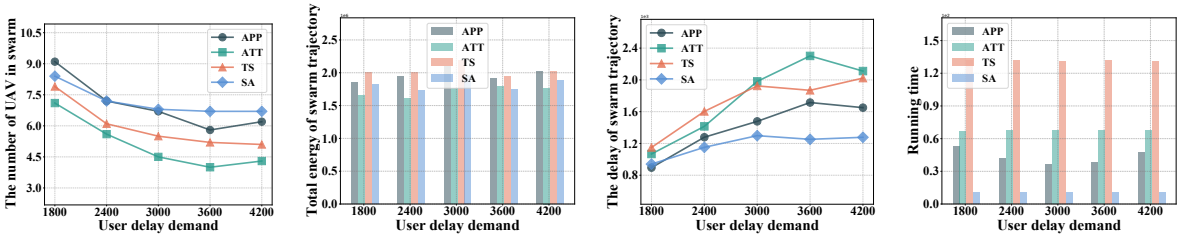
(a) The number of UAVs in the swarm (b) The total energy consumption (c) The total trajectory delay (d) The running time

Fig. 6: The performance of different schedulers by varying the UAV battery capacity from 400 to 800 kJ.



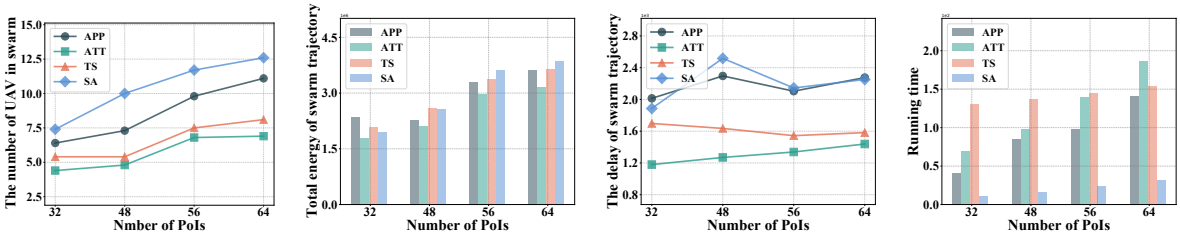
(a) The number of UAVs in the swarm (b) The total energy consumption (c) The total trajectory delay (d) The success ratio

Fig. 7: The performance of different schedulers by varying the UAV speed from 10 to 35 m/s.



(a) The number of UAVs in the swarm (b) The total energy consumption (c) The total trajectory delay (d) The running time

Fig. 8: The performance of different schedulers by varying the user delay demand from 1,800 to 4,200 s.



(a) The number of UAVs in the swarm (b) The total energy consumption (c) The average trajectory delay (d) The running time

Fig. 9: The performance of different schedulers by varying the number of PoIs.

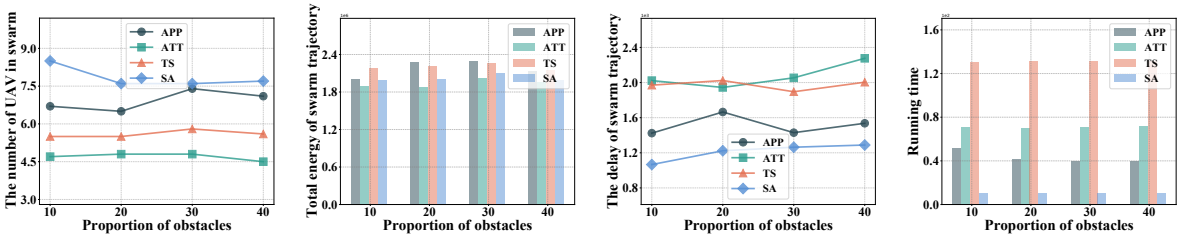
delay than TS, as shown in Fig. 6(b) and (c). Even when UAV counts differ, ATT keeps overheads bounded. Relative to APP, it may incur 0.03% more energy and 30.3% more delay while saving 41.7% UAVs, and compared to SA, it saves 11.0% energy and 89.1% UAVs at a 50.2% delay increase. Moreover, ATT maintains lower runtime than TS as battery capacity grows in Fig. 6(d).

UAV ground speed: UAV ground speed v_g nonlinearly affects system performance, reflecting a trade-off between monitoring efficiency and energy sustainability. As shown in Fig. 7(a)–(c), increasing v_g from 10 to 35 m/s first reduces UAV count and energy use, but excessive speed sharply raises per-UAV energy consumption, reversing earlier gains and increasing the required UAVs due to battery limits. ATT adapts well across all speeds, cutting UAV count by 0.7%–53.7% and energy by up to 29.6% compared with APP, 13.0%–24.5% with TS, and up to 32.0% with SA. Even when UAV count is similar to TS, ATT still reduces energy and delay by 23.4% and 17.8%, respectively. Furthermore, as shown in Fig. 7(c), monitoring delay consistently decreases with speed, benefiting from faster travel and a greater UAV count. APP achieves 30.0% lower delay than ATT but requires 43.4% more UAVs.

User delay demand: Relaxing user delay $\mathfrak{T}_i$ from 1,800

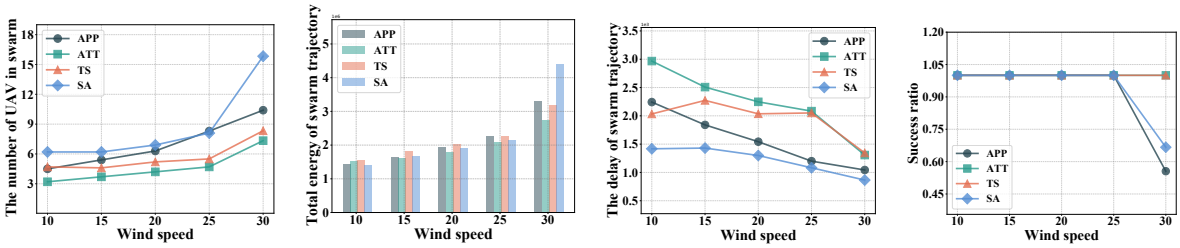
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Mobile Computing. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TMC.2026.3745584



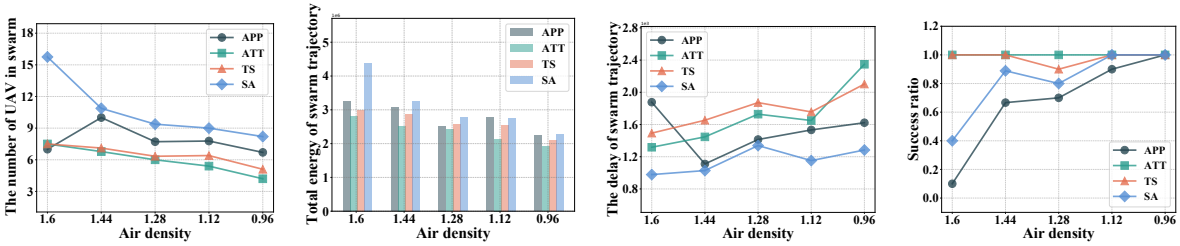
(a) The number of UAVs in the swarm (b) The total energy consumption (c) The average trajectory delay (d) The running time

Fig. 10: The performance of different schedulers by varying the proportion of obstacles.



(a) The number of UAVs in the swarm (b) The total energy consumption (c) The total trajectory delay (d) The success ratio

Fig. 11: The performance of different schedulers by varying the wind speed from 10 to 30 m/s.



(a) The number of UAVs in the swarm (b) The total energy consumption (c) The total trajectory delay (d) The success ratio

Fig. 12: The performance of different schedulers by varying the air density from 1.6 to 0.96 kg/m^3.

to 4,200 s allows each UAV to cover more PoIs, reducing swarm size but increasing energy consumption and monitoring delay. Specifically, ATT reduces UAV count by 0.9% to 67.4% as delay tolerance increases compared with others, as shown in Fig. 8(a). In terms of energy consumption, ATT initially saves up to 11.2% compared with SA, but may incur up to 0.3% more energy as UAV workload intensifies. For APP, the maximum energy decrease is around 20%, while ATT achieves a 0.8% to 25.0% energy reduction compared with TS. When UAV numbers are comparable, ATT also reduces delay by up to 13%. Regarding the monitoring delay in Fig. 8(c), other algorithms can benefit from larger UAV counts. APP achieves up to 25.4% lower delay at the cost of using 45.0% more UAVs, and SA reduces delay by up to 45.6% with 67.5% more UAVs. Finally, as shown in Fig. 8(d), delay tolerance slightly affects algorithm runtime, but ATT consistently runs faster than TS.

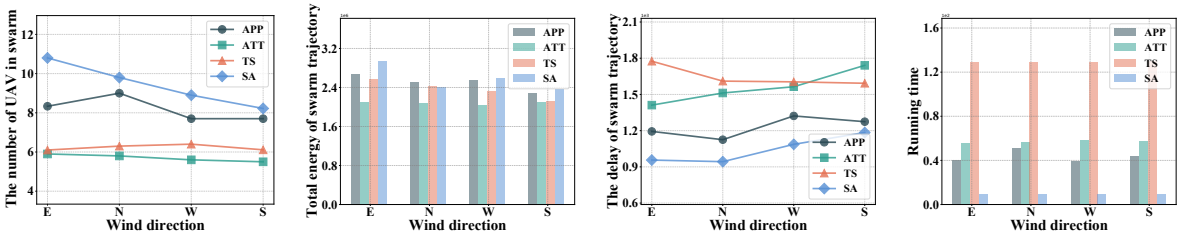
Network topology: As the number of PoIs increases from 32 to 64, the number of UAVs needed, the average monitoring delay, and the total energy consumption also increase, as shown in Fig. 9. Nevertheless, ATT consistently outperforms the baseline algorithms across all metrics. Furthermore, with the growth of the proportion of obstructed links between PoIs (due to obstacles) from 10% to 40%, ATT becomes superior to others in terms of both the UAV swarm size and the total energy consumption, as shown in Fig. 10(a) and (b), respectively. When a certain portion of links becomes unavailable, the reduction in UAV count inevitably leads to a longer inspection time. Overall, the proposed algorithm demonstrates robust performance in an obstacle-prone environment.

E. Environmental parameters for system

Wind speed: Environmental conditions act as a critical stress test that amplifies the inherent limitations of different tools. To this end, we evaluated performance across wind speeds of 10–30 m/s, revealing a nonlinear effect: higher aerodynamic resistance forces all algorithms to deploy more UAVs and consume more energy, as shown in Fig. 11. During this process of environmental deterioration, ATT adapts best, reducing UAV count by 13.5%–1.2× compared with baselines in Fig. 11(a) and saving up to 20.2% energy relative to APP under high wind, though it may use 0.5% more energy under low wind in Fig. 11(b). In contrast, TS consumes 0.1%–16.0% more energy than ATT due to less efficient routing with similar

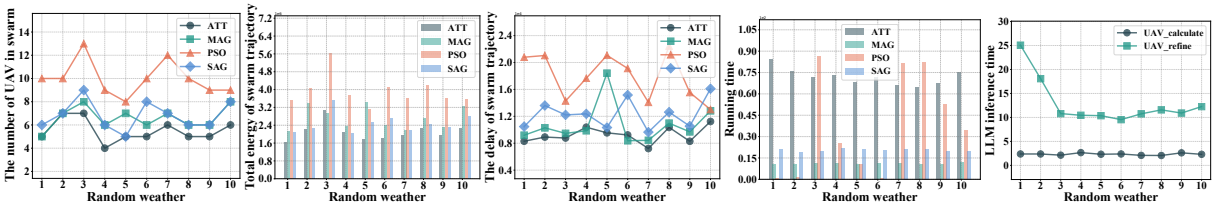
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Mobile Computing. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TMC.2026.3745584



(a) The number of UAVs in the swarm (b) The total energy consumption (c) The total trajectory delay (d) The running time

Fig. 13: The performance of different schedulers by varying the wind direction from East to South.



(a) The number of UAVs in the swarm (b) The total energy consumption (c) The average trajectory delay (d) The running time (e) The LLM inference time

Fig. 14: The performance of different schedulers by varying weather conditions.

UAV counts. Regarding delay, APP achieves up to 31.4% lower delay but requires 50.0% more UAVs, while SA reduces delay by 52.2% at the cost of 93.7% more UAVs in Fig. 11(c). Under high winds, APP and SA may even face unsolvable scenarios, as shown in Fig. 11(d). In addition, because ATT minimizes the number of UAVs, which both forces each UAV to serve more PoIs and limits depot selection flexibility whereas using more UAVs allows better depots to reduce total energy consumption, ATT occasionally has slightly higher total energy consumption than other baselines under low-wind conditions in Fig. 11(b).

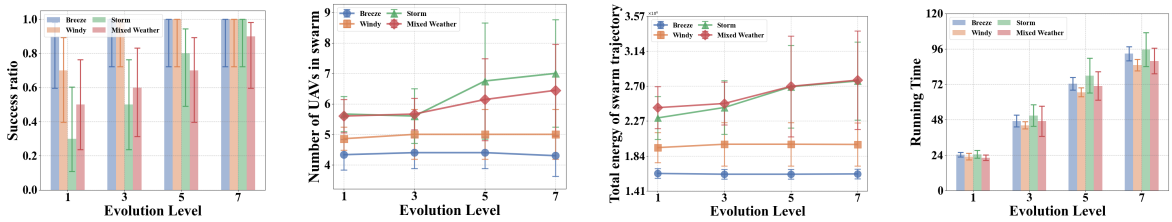
Air density: As another environmental factor, air density imposes significant operational constraints on monitoring systems. We varied ρ from 1.6 to 0.96 kg/m^3. As shown in Figs. 12(a) and (b), lower density reduces aerodynamic drag, allowing each UAV to cover more PoIs, which lowers UAV count and energy consumption but increases total monitoring delay in Fig. 12(c). Interestingly, at $\rho = 1.6$, APP has the lowest UAV count in Fig. 12(a) but the highest energy consumption in Fig. 12(b). This occurs because Fig. 12(d) shows that only one of its ten runs is successful, in which the fewest UAVs cover all PoIs over longer routes. Apart from this anomaly, ATT adapts well, cutting UAV count up to 1.1× compared with other methods. When UAV numbers are comparable (e.g., $\rho = 1.28$), ATT saves 5.6% energy and reduces delay by 6.7% versus TS, whereas APP and SA achieve larger delay reductions (30.9% and 45.3%) but require 59.5% and 95.2% more UAVs. As shown in Fig. 12(d), APP, SA, and TS always encounter infeasible scenarios when air density is high, but their performance correspondingly improves as environmental conditions improve. In addition, SA and APP become infeasible under high-wind or high-air-density scenarios because tighter energy constraints expose their algorithmic limitations. APP's greedy construction propagates early clustering errors, while SA's local search traps it in infeasible optima under limited iterations.

Wind direction: We evaluated the impact of wind direction θ_w on the performance of the proposed algorithm, by varying the direction from 0° (East) to 270° (South). As shown in Fig. 13, changes in wind direction have minimal impact on the number of required UAVs, trajectory delay, and runtime. Notably, ATT consistently consumes less energy than all other algorithms, regardless of wind direction, as shown in Fig. 13(b). While wind direction is often considered critical in UAV path planning, our findings suggest that its isolated influence on system performance is limited. This challenges prevailing assumptions and underscores the importance of evaluating wind direction in conjunction with other environmental factors such as wind speed and air density.

Random weather conditions: To account for dynamic and unpredictable weather conditions over different time slots, we incorporated both single-agent RL and multi-agent algorithms in our experiments, namely SAG and MAG, respectively. As shown in Fig. 14, while ATT occasionally incurs a slightly higher latency (e.g., slots 4 and 6) or a slightly larger total energy consumption (slot 3) in comparison with those by MAG, it still achieves the lowest average across most baselines and outperforms other RL-based methods. This is mainly attributed to the smaller number of UAVs employed by ATT. The use of fewer UAVs inevitably increases the energy consumption per UAV and the monitoring delay when all PoIs must be covered. Nevertheless, since our objective is to minimize the number of UAVs subject to delay and energy constraints, ATT is superior to other algorithms. In addition, with el trajectory-planning evaluations in Algorithm 2, the agent makes el LLM calls, comprising one initial UAV-count estimation and $el - 1$ refinements. The detailed latency is shown in Fig. 14(e), where the time for the LLM to generate the initial UAV solution is approximately 2.3 seconds, while its refinement time (also by the LLM) is about 10 seconds.

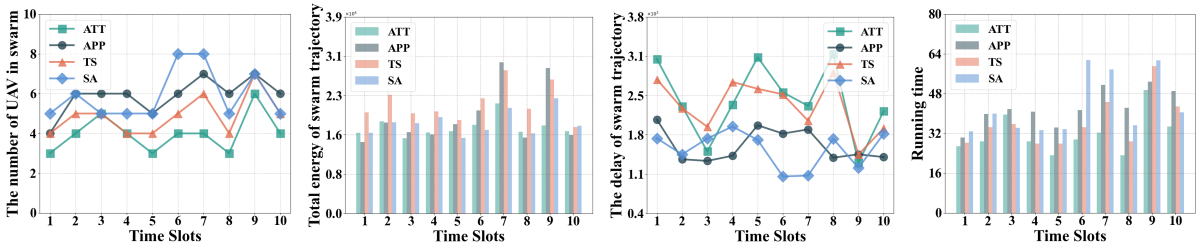
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Mobile Computing. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TMC.2026.3745584



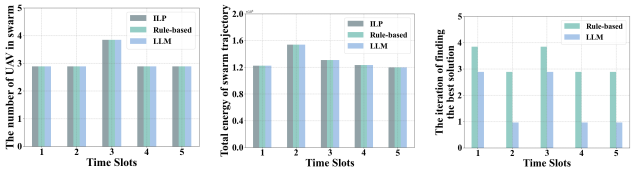
(a) The success ratio (b) The number of UAVs in the swarm (c) The total energy consumption (d) The running time

Fig. 15: The performance of the ATT algorithm by varying evolution levels from 1 to 7.



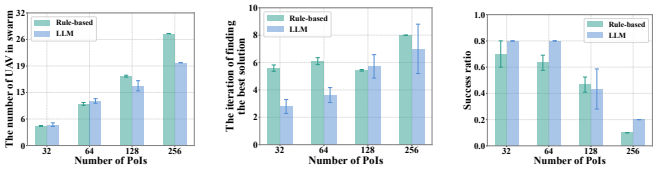
(a) The number of UAVs in the swarm (b) The total energy consumption (c) The total trajectory delay (d) The running time

Fig. 16: The performance of different schedulers by using a rule-based UAV refinement module.



(a) The number of UAVs in the swarm (b) The total energy consumption (c) The first evaluation producing the final solution

Fig. 17: Comparison of ILP with ATT using LLM-based and rule-based refinement.



(a) The number of UAVs in the swarm (b) The first evaluation producing the final solution (c) The success ratio

Fig. 18: Comparison of ATT using LLM-based and rule-based refinement over different numbers of PoIs across three seeds.

F. Ablation experiments for system

To evaluate the contribution of the UAV Count Refinement module, we used `ATT` as the UAV trajectory planner and varied the evolution level from 1 to 7 under different weather conditions. Level 1 represents no-refinement planning, whereas higher levels iteratively adjust the UAV count according to trajectory feasibility. As shown in Fig. 15, the success rate under breeze conditions increases from 90% at level 1 to 100% at level 3 and remains at 100% thereafter, whereas under windy conditions it increases from 70% at level 1 to 100% at level 3. Under storm and mixed-weather conditions across eight regions, the success rate increases from 30% and 50% at level 1 to 100% and 90% at level 7, respectively. Since an extreme weather condition simultaneously affecting all regions is uncommon in the real world, level 5 is sufficient for most scenarios and provides a practical balance among routing feasibility, the demanded number of UAVs, and running time of the proposed algorithm.

To evaluate the UAV Trajectory Planning module, we applied the same rule-based UAV Count Refinement module to `ATT`, `APP`, `TS`, and `SA`. For each of the mentioned algorithms, the UAV count is sequentially increased from 1 to 10, and an initial solution satisfying the coverage, energy, and delay constraints was selected. As shown in Fig. 16, all results are averaged over 10 time slots, and all four algorithms achieve a 100% success rate. `ATT` requires the fewest UAVs, averaging 4, and achieves the lowest total energy consumption and running time. Although `SA` delivers the lowest trajectory delay, `ATT` provides the best performance in terms of the number of UAVs, energy consumption, and computational efficiency.

We then used `ATT` as the UAV trajectory planner and compared the LLM-based refinement, rule-based refinement, and ILP algorithms in scenarios with 8 PoIs. The `ILP` provides the exact solution over the precomputed paths by lexicographically minimizing the number of UAVs and then the total energy consumption of all UAVs. As shown in Figs. 17(a) and (b), both refinement methods achieve the same optimum as the ILP over five time slots. Fig. 17(c) shows that the optimum first appears earlier with LLM-based refinement than with rule-based refinement (1.8 versus 3.4 on average), demonstrating its advantage under a limited number of planning attempts. However, increasing the number of PoIs from 8 to 32 increases the number of ILP binary variables from approximately 3,000 to 40,000, causing the solution time to exceed 1 hour and making ILP computationally impractical for large-scale appli-

© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Mobile Computing. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TMC.2026.3745584

cations. To evaluate the scalability of the proposed algorithm, we compared LLM-based and rule-based refinements using the same ATT planner for 32–256 PoIs under mixed-weather conditions across ten slots and three seeds. Both refinement methods were allowed up to ten ATT evaluations. The rule-based refinement tests UAV counts from low to high, namely, $1, 2, \ldots, 10$, $2, 4, \ldots, 20$, and $4, 8, \ldots, 40$ for 32, 64, and both 128 and 256 PoIs, respectively. For 256 PoIs, 40 is the largest multiple of four below the 42-depot deployment limit. The LLM-based refinement starts from an LLM-estimated UAV count and adjusts it using feasibility feedback.

Figs. 18(a) and (b) average only successful slots, whereas Fig. 18(c) considers all slots. Each bar is the mean of three seed-level averages, and each error bar denotes one standard deviation. Fig. 18(b) records the first evaluation that generates the final solution returned at the end of a run, and a smaller value therefore means that the method reaches its eventual solution earlier. In addition, the success ratios of the two refinement methods can differ because ATT can merge a large initial swarm into a feasible one, whereas directly using a small swarm may fail. As the number of PoIs increases, the required number of UAVs generally rises. Compared with rule-based refinement, LLM-based refinement reaches its final solution earlier at 32, 64, and 256 PoIs and requires fewer UAVs at 128 and 256 PoIs. The lower rule-based UAV averages at 32 and 64 PoIs should not be interpreted as better performance because failed slots are excluded. At 128 PoIs, the slightly earlier rule-based result in Fig. 18(b) may be aided by testing UAV counts in increments of four, although its final solution requires more UAVs. At 256 PoIs, under the 42-UAV deployment limit and ten-evaluation budget, the success ratios of the LLM-based and rule-based methods are 20% and 10%, respectively.

Finally, we conducted a component ablation for the dual-mode learning module in the Bridge Defect and Riverbed classification tasks, as shown in Tab. III. Removing Replay decreases the accuracy from 77.06% to 75.32% in Bridge Defect classification and from 97.89% to 96.84% in Riverbed classification, confirming its role in knowledge retention. Removing EWC or ASAM also reduces accuracy, with the largest degradation occurring after removing ASAM in Riverbed classification (from 97.89% to 95.26%). Progressive freezing improves accuracy while reducing the training time from 9155.4 s to 6762.5 s for Bridge Defect classification and from 1171.3 s to 767.1 s for Riverbed classification.

Dataset	Variant	Accuracy (%)	mAP (%)	Forgetting rate (%)	Training time (s)
Bridge defect classification	w/o EWC	76.11	**94.46**	0.33	6210.8
	w/o Replay	75.32	93.52	0.60	**1589.8**
	w/o ASAM	76.42	93.20	0.30	5770.7
	w/o Freezing	76.27	94.19	0.51	9155.4
	TRI (Ours)	**77.06**	94.24	**0.19**	6762.5
Riverbed classification	w/o EWC	96.32	99.61	**0.11**	766.1
	w/o Replay	96.84	99.59	0.32	**202.9**
	w/o ASAM	95.26	99.17	0.53	680.1
	w/o Freezing	96.32	99.40	0.37	1171.3
	TRI (Ours)	**97.89**	**99.63**	0.47	767.1

TABLE III: Component ablation of TRI on different classification datasets.

VI. Conclusion

In this paper, we developed an automatic hydraulic monitoring system based on an LLM agent-driven UAV swarm, by studying key challenges including dynamic weather conditions, battery limitations, and real-time learning demands in hydraulic metaverses. The system comprises three core components: an environment perception tool, an intelligent UAV scheduling tool, and a flexible model training tool that is capable of operating in both snapshot and continual learning modes. Experimental results demonstrate significant advantages of the proposed system compared with existing ones, including reductions in the number of UAVs deployed (up to $1.2\times$), energy consumption (up to 29.6%), and the average monitoring delay (up to 18.4%), as well as improvements in detection accuracy. Our findings show that LLM agents can effectively manage UAV swarm operations for hydraulic engineering monitoring while adapting to environmental variability.

Several issues that arose from this study are worth further investigation in the future. This work may constrain potential LLM hallucinations through trajectory planning and solution feasibility checks, while alleviating data bias and forgetting through imbalance-aware learning and historical knowledge preservation. Nevertheless, privacy risks in captured imagery require further consideration. Moreover, vision-language models (VLMs) will be explored to interpret real-time onboard imagery and environmental conditions, enabling fine-grained trajectory adjustment.

Acknowledgment

The work by Min Chen was supported by the National Key R&D Program of China (2025YFE0213400). The work by Weifa Liang was supported by grants from the Research Grants Council of the Hong Kong Special Administrative Region, China, under Grant Nos. CityU 11202723, CityU 11202824, C1042-23GF, 8730094, and 9380137.

References

[1] R. P. Alexsah and A. A. S. Gunawan, "Re-enhancing multi-target classification techniques for CODEBRIM dataset using deep learning algorithm," in *2024 IEEE International Conference on Communication, Networks and Satellite (COMNETSAT)*, 2024, pp. 90–95.

[2] J. Alotaibi, O. S. Oubbati, M. Atiquzzaman, F. Alromithy, and M. R. Altimania, "Optimizing disaster response with UAV-mounted RIS and HAP-enabled edge computing in 6G networks," *Journal of Network and Computer Applications*, vol. 241, p. 104213, 2025.

[3] A. I. Ameur, O. S. Oubbati, A. Rachedi, A. Arishi, and M. Atiquzzaman, "Intelligent UAV caching and energy management in 6G networks," *IEEE Transactions on Network Science and Engineering*, 2025.

[4] M. Bakirci, P. Dmytrovych, I. Bayraktar, and O. Anatoliyovych, "Multi-class vehicle detection and classification with YOLO11 on UAV-captured aerial imagery," in *2024 IEEE 7th International Conference on Actual Problems of Unmanned Aerial Vehicles Development (APUAVD)*. IEEE, 2024, pp. 191–196.

[5] S.-W. Chang, J.-J. Kuo, M.-J. Kao, B.-Z. Chen, and Q.-J. Wang, "Near-optimal UAV deployment for delay-bounded data collection in IoT networks," in *IEEE INFOCOM 2024 - IEEE Conference on Computer Communications*, 2024, pp. 111–120.

[6] A. Che, C. Wang, K. Lu, T. Tao, W. Wang, and B. Wang, "Multi-label classification for concrete defects based on EfficientNetV2," in *Advanced Intelligent Computing Technology and Applications: 20th International Conference, ICIC 2024, Tianjin, China, August 5–8, 2024, Proceedings, Part IV*. Berlin, Heidelberg: Springer-Verlag, 2024, pp. 37–48.

© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Mobile Computing. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TMC.2026.3745584

[7] M. Chen, Y. Wang, L. Ai, X. Cheng, J. Xie, H. Yi, E. Lan, J. Liu, J. Luo, Y. Qiu, R. Wang, X.-K. Wu, W. Xiao, Y. Hao, L. Hu, D. Zeng, Z. He, G. Wei, S. Liu, F. Zhang, and H. Tang, "A hydro-intelligence body: Cognitive architecture for project-oriented autonomous water systems driven by multi-modal sensing and water utility panel," *IEEE Transactions on Consumer Electronics*, vol. 72, no. 1, pp. 1948–1957, 2026.

[8] M. De Lange, R. Aljundi, M. Masana, S. Parisot, X. Jia, A. Leonardis, G. Slabaugh, and T. Tuytelaars, "A continual learning survey: Defying forgetting in classification tasks," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 7, pp. 3366–3385, 2022.

[9] L. Deng, W. Xu, W. Liang, J. Peng, Y. Zhou, L. Duan, and S. K. Das, "Approximation algorithms for the min-max cycle cover problem with neighborhoods," *IEEE/ACM Transactions on Networking*, vol. 28, no. 4, pp. 1845–1858, 2020.

[10] X. Du, Y. Sun, Y. Song, L. Dong, C. Tao, and D. Wang, "Recognition of underwater engineering structures using CNN models and data expansion on side-scan sonar images," *Journal of Marine Science and Engineering*, vol. 13, no. 3, p. 424, 2025.

[11] X. Du, Y. Sun, Y. Song, L. Dong, and X. Zhao, "Revealing the potential of deep learning for detecting submarine pipelines in side-scan sonar images: An investigation of pre-training datasets," *Remote Sensing*, vol. 15, no. 19, p. 4873, 2023.

[12] T. Fan, L. Fu, C. Guo, Y. Zhang, and L. Sun, "Multi-UAV inspection optimization for offshore wind farms considering battery exchange process," *IEEE Transactions on Intelligent Vehicles*, pp. 1–11, 2024.

[13] S. Gong, M. Wang, B. Gu, W. Zhang, D. T. Hoang, and D. Niyato, "Bayesian optimization enhanced deep reinforcement learning for trajectory planning and network formation in multi-UAV networks," *IEEE Transactions on Vehicular Technology*, vol. 72, no. 8, pp. 10 933–10 948, 2023.

[14] Q. Guo, J. Peng, W. Xu, W. Liang, X. Jia, Z. Xu, Y. Yang, and M. Wang, "Minimizing the longest tour time among a fleet of UAVs for disaster area surveillance," *IEEE Transactions on Mobile Computing*, vol. 21, no. 7, pp. 2451–2465, 2022.

[15] A. R. Hileman, "Weather and its effect on air insulation specifications," *IEEE Transactions on Power Apparatus and Systems*, vol. PAS-103, no. 10, pp. 3104–3116, 1984.

[16] Q. Huang, Z. Sheng, Y. Fang, and J. Li, "A simulated annealing-particle swarm optimization algorithm for UAV multi-target path planning," in *2022 2nd International Conference on Consumer Electronics and Computer Engineering (ICCECE)*, 2022, pp. 906–910.

[17] R. Jia, Q. Fu, Z. Zheng, G. Zhang, and M. Li, "Energy and time trade-off optimization for multi-UAV enabled data collection of IoT devices," *IEEE/ACM Transactions on Networking*, vol. 32, no. 6, pp. 5172–5187, 2024.

[18] J. Kirkpatrick, R. Pascanu, N. Rabinowitz, J. Veness, G. Desjardins, A. A. Rusu, K. Milan, J. Quan, T. Ramalho, A. Grabska-Barwinska, D. Hassabis, C. Clopath, D. Kumaran, and R. Hadsell, "Overcoming catastrophic forgetting in neural networks," *Proceedings of the National Academy of Sciences*, vol. 114, no. 13, pp. 3521–3526, 2017.

[19] J. Li, S. Guo, W. Liang, J. Wang, Q. Chen, Y. Zeng, B. Ye, and X. Jia, "Digital twin-enabled service provisioning in edge computing via continual learning," *IEEE Transactions on Mobile Computing*, vol. 23, no. 6, pp. 7335–7350, 2024.

[20] S. Li, Z. Liu, M. Zheng, and Y. Li, "Research on intelligent monitoring system of hydraulic engineering based on digital twin," in *2021 7th International Conference on Hydraulic and Civil Engineering & Smart Water Conservancy and Intelligent Disaster Reduction Forum (ICHCE & SWIDR)*, 2021, pp. 224–231.

[21] X. Li and B. Tang, "MGSER-SAM: Memory-guided soft experience replay with sharpness-aware optimization for enhanced continual learning," in *2024 International Joint Conference on Neural Networks (IJCNN)*, 2024, pp. 1–8.

[22] Y. Li, W. Liang, W. Xu, Z. Xu, X. Jia, Y. Xu, and H. Kan, "Data collection maximization in IoT-sensor networks via an energy-constrained UAV," *IEEE Transactions on Mobile Computing*, vol. 22, no. 1, pp. 159–174, 2021.

[23] Y. Li, W. Liang, Z. Xu, W. Xu, and X. Jia, "Budget-constrained digital twin synchronization and its application on fidelity-aware queries in edge computing," *IEEE Transactions on Mobile Computing*, vol. 24, no. 1, pp. 165–182, 2025.

[24] D. Lopez-Paz and M. Ranzato, "Gradient episodic memory for continual learning," ser. NIPS'17. Curran Associates Inc., 2017, pp. 6470–6479.

[25] M. Mundt, S. Majumder, S. Murali, P. Panetsos, and V. Ramesh, "Meta-learning convolutional neural architectures for multi-target concrete defect classification with the concrete defect bridge image dataset," in *2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 11 188–11 197.

[26] O. S. Oubbati, J. Alotaibi, F. Alromithy, M. Atiquzzaman, and M. R. Altimania, "A UAV-UGV cooperative system: Patrolling and energy management for urban monitoring," *IEEE Transactions on Vehicular Technology*, vol. 74, no. 9, pp. 13 521–13 536, 2025.

[27] Z. Peng, Y. Qiu, and G. Wang, "Model caching and application offloading for mobile edge intelligence network with learning-and-optimization approach," *IEEE Transactions on Services Computing*, 2025.

[28] Y. Qi, Y. He, X. Qi, Y. Zhang, and G. Yang, "Dynamic snake convolution based on topological geometric constraints for tubular structure segmentation," in *2023 IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023, pp. 6047–6056.

[29] Y. Qiu, M. Chen, W. Liang, L. Ai, and D. Niyato, "Information sharing in multi-tenant metaverse via intent-driven multicasting," *IEEE Transactions on Computers*, 2025.

[30] Y. Qiu, M. Chen, W. Liang, L. Ai, D. Niyato, and G. Wei, "Privacy-enhanced healthcare monitoring service refreshment in human digital twin-assisted fabric metaverse," *IEEE Transactions on Mobile Computing*, 2025.

[31] J. C. SanMiguel and A. Cavallaro, "Energy consumption models for smart camera networks," *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 27, no. 12, pp. 2661–2674, 2017.

[32] F. Shan, J. Huang, R. Xiong, F. Dong, J. Luo, and S. Wang, "Energy-efficient general PoI-visiting by UAV with a practical flight energy model," *IEEE Transactions on Mobile Computing*, vol. 22, no. 11, pp. 6427–6444, 2023.

[33] L. Shi, Y. Gao, C. Chen, S. Huang, J. Zhao, and X. Hu, "Privacy-preserving rényi layer-wise budget allocation against gradient leakage for federated learning," *IEEE Transactions on Mobile Computing*, 2025.

[34] F. B. Sorbelli, F. Corò, S. K. Das, and C. M. Pinotti, "Energy-constrained delivery of goods with drones under varying wind conditions," *IEEE Transactions on Intelligent Transportation Systems*, vol. 22, no. 9, pp. 6048–6060, 2021.

[35] F. B. Sorbelli, F. Corò, L. Palazzetti, C. M. Pinotti, and G. Rigoni, "How the wind can be leveraged for saving energy in a truck-drone delivery system," *IEEE Transactions on Intelligent Transportation Systems*, vol. 24, no. 4, pp. 4038–4049, 2023.

[36] X. Tang, F. Liu, D. Xu, J. Jiang, Q. Tang, B. Wang, Q. Wu, and C. L. Philip Chen, "LLM-assisted reinforcement learning: Leveraging lightweight large language model capabilities for efficient task scheduling in multi-cloud environment," *IEEE Transactions on Consumer Electronics*, vol. 71, no. 2, pp. 5631–5644, 2025.

[37] J. S. Vitter, "Random sampling with a reservoir," *ACM Trans. Math. Softw.*, vol. 11, no. 1, pp. 37–57, Mar. 1985.

[38] Z. Wang, T. Wei, G. Sun, X. Liu, H. Yu, and D. Niyato, "Multi-UAV enabled MEC networks: Optimizing delay through intelligent 3-d trajectory planning and resource allocation," *IEEE Transactions on Intelligent Transportation Systems*, 2025.

[39] M. Wei, Y. Yu, X. Du, Y. Song, L. Dong, Q. Zhou, L. Wang, L. Zhang, and Y. Wang, "Automated detection of submarine pipelines in the yellow river estuary: a deep learning approach for side-scan sonar data in dynamic deltaic systems," *Frontiers in Earth Science*, vol. 13, p. 1596238, 2025.

[40] D. Wu, X. Wang, Y. Qiao, Z. Wang, J. Jiang, S. Cui, and F. Wang, "NetLLM: Adapting large language models for networking," in *Proceedings of the ACM SIGCOMM 2024 Conference*, ser. ACM SIGCOMM '24. New York, NY, USA: Association for Computing Machinery, 2024, pp. 661–678.

[41] J. Wu, Y. Sun, J. Bi, and C. Chen, "A novel hybrid enhanced particle swarm optimization for UAV path planning," *IEEE Transactions on Vehicular Technology*, 2025.

[42] Q. Wu, Y. Xie, P. Fan, D. Qin, K. Wang, N. Cheng, and K. B. Letaief, "Large language model-based task offloading and resource allocation for digital twin edge computing networks," *IEEE Transactions on Mobile Computing*, 2026.

[43] W. Xu, Y. Sun, R. Zou, W. Liang, Q. Xia, F. Shan, T. Wang, X. Jia, and Z. Li, "Throughput maximization of UAV networks," *IEEE/ACM Transactions on Networking*, vol. 30, no. 2, pp. 881–895, 2022.

[44] W. Xu, C. Wang, H. Xie, W. Liang, H. Dai, Z. Xu, Z. Wang, B. Guo, and S. K. Das, "Reward maximization for disaster zone monitoring with heterogeneous UAVs," *IEEE/ACM Transactions on Networking*, vol. 32, no. 1, pp. 890–903, 2024.

[45] Y. Zeng, J. Xu, and R. Zhang, "Energy minimization for wireless communication with rotary-wing UAV," *IEEE Transactions on Wireless Communications*, vol. 18, no. 4, pp. 2329–2345, 2019.

© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.